

DOKUMEN DESAIN SISTEM
LIBRARY BOOK BORROWING SYSTEM

Dosen Pengampu: Ike Pertiwi Windasari, S.T., M.T.



Disusun oleh:

Lanjar Setiawan	21120124120023
Robertus William Susanto	21120124120007

DEPARTEMEN TEKNIK KOMPUTER
FAKULTAS TEKNIK
UNIVERSITAS DIPONEGORO
SEMARANG
2026

BAB I

PENDAHULUAN

1.1. Latar Belakang Sistem

Dalam kehidupan akademik, mahasiswa sering dihadapkan pada berbagai tugas dari beberapa mata kuliah dengan tenggat waktu yang berbeda-beda. Kondisi tersebut menyebabkan mahasiswa perlu memiliki kemampuan manajemen waktu yang baik agar seluruh tugas dapat diselesaikan secara tepat waktu. Namun, pada kenyataannya masih banyak mahasiswa yang mengalami kesulitan dalam mengatur prioritas pekerjaan, mencatat tugas secara terstruktur, serta memantau progres penyelesaian tugas yang sedang dikerjakan. Akibatnya, beberapa tugas dapat terlewat, terlambat dikumpulkan, atau bahkan terlupakan karena kurangnya sistem pengelolaan yang efektif.

Sebagian besar mahasiswa masih menggunakan metode konvensional dalam mengelola tugas, seperti mencatat pada buku catatan, aplikasi pesan pribadi, maupun hanya mengandalkan ingatan. Pendekatan tersebut sering kali kurang efektif karena tidak memberikan gambaran yang jelas mengenai daftar tugas, tingkat prioritas, maupun progres penyelesaiannya. Selain itu, ketika jumlah tugas meningkat dalam waktu bersamaan, mahasiswa menjadi lebih sulit menentukan pekerjaan mana yang harus diselesaikan terlebih dahulu berdasarkan tingkat urgensi.

Untuk mengatasi permasalahan tersebut, diperlukan suatu sistem yang mampu membantu pengguna dalam mencatat, mengelola, dan memantau tugas secara lebih terstruktur. Salah satu solusi yang dapat diterapkan adalah penggunaan aplikasi berbasis *To-Do List* yang memungkinkan pengguna menyimpan daftar pekerjaan secara sistematis dalam satu platform terintegrasi. Sistem ini tidak hanya berfungsi sebagai media pencatatan tugas, tetapi juga membantu pengguna dalam mengorganisasi pekerjaan berdasarkan prioritas tertentu.

Berdasarkan kebutuhan tersebut, dikembangkan sebuah sistem manajemen tugas berbasis web bernama "DoneRight". Sistem ini dirancang untuk membantu mahasiswa dalam mengelola tugas akademik secara lebih efektif melalui fitur pencatatan tugas, pengelompokan kategori, penentuan skala prioritas (*high, medium, low*), pengaturan *deadline*, pencarian tugas, penyaringan status tugas, serta visualisasi progres kerja. Dengan adanya fitur-fitur tersebut, pengguna diharapkan dapat lebih mudah memahami daftar pekerjaan yang harus diselesaikan, menentukan tugas yang paling mendesak, serta memantau perkembangan penyelesaian tugas secara keseluruhan.

Selain itu, sistem DoneRight dikembangkan dalam bentuk aplikasi berbasis web agar dapat diakses secara fleksibel melalui *browser* tanpa memerlukan instalasi tambahan pada perangkat pengguna. Dengan pendekatan tersebut, sistem diharapkan mampu memberikan kemudahan akses, meningkatkan efektivitas pengelolaan tugas, serta membantu mahasiswa dalam membangun kebiasaan manajemen waktu yang lebih baik.

1.2. Tujuan Dokumen

Dokumen Desain Sistem (D2) ini disusun sebagai bentuk lanjutan dari dokumen *Software Requirements Specification* (SRS/D1) yang telah dibuat sebelumnya. Tujuan utama dokumen ini adalah untuk menerjemahkan kebutuhan sistem yang telah diidentifikasi pada tahap analisis ke dalam bentuk rancangan sistem yang lebih terstruktur dan sistematis sebelum memasuki tahap implementasi perangkat lunak.

Secara khusus, dokumen ini bertujuan untuk:

1. Menjelaskan rancangan sistem aplikasi DoneRight berdasarkan kebutuhan fungsional dan non-fungsional yang telah ditetapkan pada dokumen SRS (D1).
2. Menyediakan representasi visual sistem menggunakan pendekatan Unified Modeling Language (UML), seperti *Use Case Diagram*, *Activity Diagram*, *Sequence Diagram*, dan *Class Diagram* untuk menggambarkan alur kerja serta struktur sistem.
3. Menjelaskan rancangan struktur data melalui Entity Relationship Diagram (ERD) sebagai dasar pengelolaan data pada sistem.
4. Mendeskripsikan arsitektur sistem secara sederhana yang meliputi hubungan antara komponen *frontend*, *backend*, dan *database* dalam mendukung operasional aplikasi.
5. Menyusun rancangan antarmuka pengguna (*wireframe*) dan prototipe *low-fidelity* guna memberikan gambaran mengenai tampilan serta alur interaksi pengguna dengan sistem.
6. Menjadi acuan utama dalam proses implementasi sistem pada tahap pengembangan selanjutnya sehingga pengembangan dapat dilakukan secara lebih terarah, konsisten, dan sesuai kebutuhan pengguna.

Dengan adanya dokumen desain sistem ini, seluruh anggota tim pengembang diharapkan memiliki pemahaman yang sama mengenai rancangan sistem yang akan dibangun sehingga proses implementasi dapat berjalan lebih efektif dan meminimalkan kesalahan pengembangan.

1.3. Ruang Lingkup Sistem

Ruang lingkup sistem menjelaskan batasan fitur dan fungsi yang termasuk dalam pengembangan aplikasi **DoneRight** pada tahap implementasi saat ini. Sistem yang dikembangkan merupakan aplikasi manajemen tugas berbasis web yang berfokus pada pengelolaan tugas akademik mahasiswa secara individual.

Aplikasi DoneRight menyediakan berbagai fitur utama yang mendukung proses pengelolaan tugas, di antaranya:

1. Pembuatan tugas baru, yaitu pengguna dapat menambahkan tugas dengan memasukkan deskripsi tugas, kategori, tingkat prioritas, serta deadline.
2. Pengelolaan daftar tugas, yang memungkinkan pengguna melihat, mengedit, menandai tugas sebagai selesai, dan menghapus tugas yang sudah tidak diperlukan.
3. Pengaturan prioritas tugas, menggunakan tingkat prioritas high, medium, dan low untuk membantu pengguna menentukan tingkat urgensi suatu pekerjaan.
4. Pengelompokan kategori tugas, seperti kategori pekerjaan atau pribadi untuk membantu pengguna membedakan jenis aktivitas yang dilakukan.
5. Penyaringan dan pencarian tugas, sehingga pengguna dapat menemukan tugas tertentu berdasarkan status maupun kata kunci secara lebih cepat.
6. Visualisasi progres kerja, berupa widget progres yang menampilkan jumlah tugas selesai, total tugas, dan persentase penyelesaian pekerjaan.
7. Penyimpanan data tugas, agar data tetap tersedia ketika pengguna kembali mengakses aplikasi.

Meskipun demikian, sistem ini memiliki beberapa batasan pengembangan. Pada tahap saat ini, aplikasi belum mendukung fitur autentikasi pengguna (*login/register*), kolaborasi multi-user, integrasi kalender eksternal, maupun notifikasi otomatis terkait *deadline* tugas. Sistem dirancang untuk digunakan oleh satu pengguna pada perangkat yang sama sesuai kebutuhan dasar pengelolaan tugas akademik mahasiswa.

Dengan batasan tersebut, pengembangan sistem tetap difokuskan pada kebutuhan inti pengguna agar aplikasi dapat berjalan secara optimal dan sesuai dengan tujuan awal perancangan sistem.

1.4. Referensi (mengacu ke SRS/D1)

Dokumen Desain Sistem (D2) ini disusun dengan mengacu pada beberapa referensi yang relevan sebagai dasar perancangan sistem. Referensi utama yang digunakan adalah

dokumen Software Requirements Specification (SRS/D1) yang telah disusun sebelumnya sebagai hasil analisis kebutuhan sistem aplikasi DoneRight.

Adapun referensi yang digunakan dalam penyusunan dokumen ini adalah sebagai berikut:

1. Dokumen Software Requirements Specification (SRS/D1) berjudul “*DoneRight: Pengembangan Sistem Manajemen Tugas Berbasis Web dengan Fitur Skala Prioritas dan Visualisasi Progres Kerja*” yang disusun oleh Kelompok 5 sebagai acuan utama kebutuhan sistem.
2. Standar dan konsep Unified Modeling Language (UML) sebagai metode pemodelan sistem untuk menggambarkan struktur dan perilaku sistem melalui diagram.
3. Konsep Entity Relationship Diagram (ERD) sebagai dasar dalam perancangan struktur basis data sistem.
4. Referensi terkait metode pengembangan perangkat lunak Scrum yang digunakan pada tahap implementasi sistem sebagaimana telah dijelaskan pada dokumen SRS (D1).

Dokumen ini disusun agar tetap konsisten dengan kebutuhan, tujuan, dan ruang lingkup sistem yang telah didefinisikan pada tahap analisis sehingga dapat digunakan sebagai dasar utama dalam proses implementasi perangkat lunak pada tahap selanjutnya.

BAB II

RINGKASAN SISTEM (SYSTEM OVERVIEW)

2.1 Deskripsi Umum Sistem

DoneRight merupakan aplikasi berbasis web yang dirancang untuk membantu pengguna dalam mengelola aktivitas dan tugas sehari-hari secara lebih terstruktur dan efisien. Sistem ini memungkinkan pengguna untuk mencatat, mengelola, mengelompokkan, serta memantau progres penyelesaian tugas melalui antarmuka berbasis checklist yang sederhana dan mudah digunakan.

Dalam sistem ini, pengguna diwajibkan melakukan proses registrasi dan login sebelum dapat mengakses fitur utama aplikasi. Setiap pengguna memiliki akun pribadi sehingga data tugas yang dimiliki bersifat individual dan tidak dapat diakses oleh pengguna lain. Dengan pendekatan multi-user berbasis autentikasi, sistem mampu menyimpan data secara aman dan terorganisasi menggunakan basis data terpusat.

Fitur utama sistem meliputi pengelolaan tugas (*task management*) yang terdiri atas penambahan, pengeditan, penghapusan, pencarian, pengelompokan, dan penandaan status penyelesaian tugas. Kategori tugas pada sistem dikelola oleh admin sebagai data referensi (*master category*) yang dapat digunakan oleh seluruh mahasiswa saat membuat atau mengedit tugas. Dengan pendekatan ini, sistem menjaga konsistensi pengelompokan tugas dan memudahkan proses monitoring data secara keseluruhan, sehingga sistem menjadi lebih fleksibel dan personal.

Setiap tugas memiliki atribut seperti judul tugas, deskripsi, kategori, prioritas, tenggat waktu (*deadline*), serta status penyelesaian berbasis checkbox. Sistem menerapkan mekanisme *soft delete* pada data tugas, sehingga tugas yang dihapus tidak langsung hilang secara permanen dari basis data untuk meminimalkan risiko kehilangan data akibat kesalahan pengguna.

Selain pengguna biasa (mahasiswa), sistem juga menyediakan akun dengan peran admin yang berfungsi untuk melakukan pemantauan (*monitoring*) terhadap aktivitas sistem secara umum, seperti jumlah pengguna dan jumlah tugas yang tersimpan. Selain pengguna biasa (mahasiswa), sistem juga menyediakan akun dengan peran admin yang berfungsi untuk melakukan monitoring terhadap aktivitas sistem secara umum, termasuk melihat keseluruhan data tugas lintas pengguna, memantau tugas yang melewati tenggat waktu (*overdue*), menganalisis statistik produktivitas, serta mengelola kategori tugas. Namun, admin tidak memiliki hak untuk mengubah isi tugas pribadi pengguna.

Sistem ini dikembangkan sebagai aplikasi berbasis web (*web-based application*) yang dapat diakses melalui browser tanpa memerlukan instalasi tambahan pada perangkat pengguna.

2.2 Stakeholder dan Aktor Sistem

Dalam pengembangan sistem DoneRight, terdapat beberapa pihak yang memiliki kepentingan terhadap sistem (*stakeholder*) dan pihak yang secara langsung berinteraksi dengan sistem (*aktor sistem*). Identifikasi stakeholder dan aktor bertujuan untuk memastikan bahwa sistem yang dikembangkan mampu memenuhi kebutuhan pengguna serta berjalan sesuai tujuan pengembangan.

a. Stakeholder

Stakeholder merupakan pihak yang memiliki keterlibatan atau kepentingan terhadap proses pengembangan maupun penggunaan sistem DoneRight. Adapun stakeholder pada sistem ini meliputi:

1. Mahasiswa (Pengguna Utama)

Mahasiswa merupakan stakeholder utama sekaligus target pengguna sistem. Mahasiswa menggunakan aplikasi DoneRight untuk mencatat, mengatur, serta memantau tugas akademik maupun aktivitas pribadi. Sistem ini diharapkan dapat membantu mahasiswa mengelola pekerjaan secara lebih terorganisir, menentukan prioritas tugas, dan meningkatkan efektivitas penyelesaian pekerjaan.

2. Tim Pengembang (*Developer*)

Tim pengembang merupakan pihak yang bertanggung jawab terhadap proses analisis kebutuhan, perancangan sistem, implementasi, pengujian, hingga pemeliharaan aplikasi DoneRight. Tim pengembang memiliki tanggung jawab untuk memastikan bahwa sistem berjalan sesuai kebutuhan pengguna dan memenuhi spesifikasi yang telah ditentukan pada dokumen *Software Requirements Specification* (SRS).

3. Dosen Pengampu Mata Kuliah

Dosen pengampu berperan sebagai stakeholder akademik yang memberikan arahan, evaluasi, serta masukan terhadap proses pengembangan perangkat lunak. Selain itu, dosen juga berperan dalam melakukan validasi terhadap kesesuaian penerapan konsep rekayasa perangkat lunak pada sistem yang dikembangkan.

b. Aktor Sistem

Aktor merupakan pihak yang berinteraksi langsung dengan sistem untuk menjalankan fungsi tertentu. Pada sistem DoneRight, terdapat dua aktor utama, yaitu:

1. Mahasiswa

Mahasiswa merupakan aktor utama yang menggunakan sistem untuk mengelola tugas pribadi, baik tugas akademik maupun aktivitas lainnya. Setiap mahasiswa wajib melakukan registrasi dan login sebelum dapat mengakses fitur utama sistem. Hak akses mahasiswa meliputi:

- Melakukan registrasi akun
- Login ke sistem
- Menambahkan tugas baru
- Mengedit tugas
- Menghapus tugas
- Menandai tugas selesai melalui checklist
- Mengelola kategori tugas pribadi
- Melihat daftar tugas
- Menentukan deadline tugas
- Mencari tugas
- Memfilter dan mengurutkan tugas
- Melihat progres penyelesaian tugas
- Melihat detail tugas

2. Admin

Admin merupakan aktor dengan hak akses khusus yang bertugas melakukan monitoring terhadap sistem secara umum. Hak akses admin meliputi:

- Login ke sistem
- Monitoring seluruh tugas mahasiswa
- Monitoring tugas overdue
- Monitoring statistik produktivitas
- Mengelola kategori tugas
- Generate laporan sistem

Admin tidak memiliki hak untuk mengubah isi tugas pribadi mahasiswa, melainkan hanya melakukan pemantauan dan pengelolaan data pendukung sistem.

2.3 Ringkasan Kebutuhan Fungsional (dari SRS)

Berdasarkan dokumen *Software Requirements Specification* (SRS/D1), sistem DoneRight memiliki sejumlah kebutuhan fungsional yang telah disesuaikan dengan

pengembangan desain sistem pada tahap D2. Penyesuaian dilakukan untuk mendukung implementasi sistem berbasis multi-user dengan autentikasi akun, penyimpanan basis data, serta pengelolaan tugas yang lebih fleksibel dan terstruktur. Berikut merupakan ringkasan kebutuhan fungsional sistem:

Kode	Kebutuhan Fungsional
FR-01	Sistem dapat melakukan registrasi akun pengguna
FR-02	Sistem dapat melakukan autentikasi login pengguna
FR-03	Sistem dapat menambahkan tugas baru
FR-04	Sistem dapat mengedit data tugas
FR-05	Sistem dapat menghapus tugas menggunakan mekanisme <i>soft delete</i>
FR-06	Sistem dapat menampilkan daftar tugas pengguna
FR-07	Sistem dapat menandai tugas selesai atau belum selesai melalui checkbox
FR-08	Sistem dapat melakukan pencarian tugas berdasarkan kata kunci
FR-09	Sistem dapat melakukan filter tugas berdasarkan status penyelesaian
FR-10	Sistem dapat melakukan pengurutan tugas berdasarkan prioritas atau tenggat waktu (<i>deadline</i>)
FR-11	Sistem dapat menampilkan progres penyelesaian tugas
FR-12	Sistem dapat mengelola kategori tugas milik pengguna (<i>create</i> dan <i>delete category</i>)
FR-13	Sistem dapat menampilkan detail tugas melalui fitur dropdown/detail panel
FR-14	Sistem dapat menyimpan data pengguna, kategori, dan tugas ke basis data
FR-15	Sistem dapat mengelompokkan tugas berdasarkan kategori
FR-16	Sistem memberikan pilihan saat kategori dihapus, yaitu menghapus seluruh tugas terkait atau mempertahankan tugas tanpa kategori
FR-17	Sistem menyediakan fitur monitoring sederhana untuk admin
FR-18	Sistem dapat mengelola kategori tugas oleh admin
FR-19	Sistem dapat melakukan monitoring seluruh tugas
FR-20	Sistem dapat melakukan monitoring tugas overdue
FR-21	Sistem dapat menampilkan statistik produktivitas
FR-22	Sistem dapat menghasilkan laporan sistem

Kebutuhan fungsional tersebut dirancang untuk mendukung proses manajemen tugas yang lebih terorganisasi, fleksibel, dan mudah digunakan oleh pengguna, khususnya mahasiswa dalam mengelola tugas akademik maupun aktivitas pribadi.

2.4 Batasan dan Asumsi Sistem

Dalam pengembangan sistem DoneRight, terdapat beberapa batasan dan asumsi yang digunakan agar ruang lingkup sistem tetap terarah, realistis, serta sesuai dengan kebutuhan pengguna.

2.4.1 Batasan Sistem

1. Sistem hanya dapat diakses oleh pengguna yang telah melakukan registrasi dan login.
2. Setiap pengguna hanya dapat mengakses dan mengelola data tugas miliknya sendiri.
3. Sistem tidak mendukung kolaborasi antar pengguna dalam pengelolaan tugas.
4. Prioritas tugas bersifat tetap (*fixed*) dan hanya terdiri atas tiga tingkat, yaitu: High, Medium, dan Low
5. Sistem tidak mendukung fitur pengingat otomatis (*notification/reminder*) berbasis email maupun notifikasi perangkat.
6. Penghapusan tugas menerapkan mekanisme *soft delete*, sehingga data tidak langsung dihapus secara permanen dari basis data.
7. Pengguna dapat membuat kategori tugas secara mandiri sesuai kebutuhan masing-masing.
8. Penghapusan kategori memberikan dua pilihan kepada pengguna, yaitu: Menghapus seluruh tugas yang terkait dengan kategori tersebut, dan mempertahankan tugas dengan menghapus hubungan kategori (*category_id = NULL*)
9. Sistem menampilkan detail tugas menggunakan fitur dropdown/detail panel tanpa berpindah halaman.
10. Sistem berbasis web (*web-based*) dan membutuhkan koneksi internet untuk mengakses data.
11. Admin memiliki hak akses monitoring terhadap data tugas lintas pengguna, namun tidak dapat mengubah isi tugas pribadi mahasiswa.

2.4.2 Asumsi Sistem

1. Pengguna memiliki perangkat yang mendukung browser modern serta koneksi internet yang stabil.

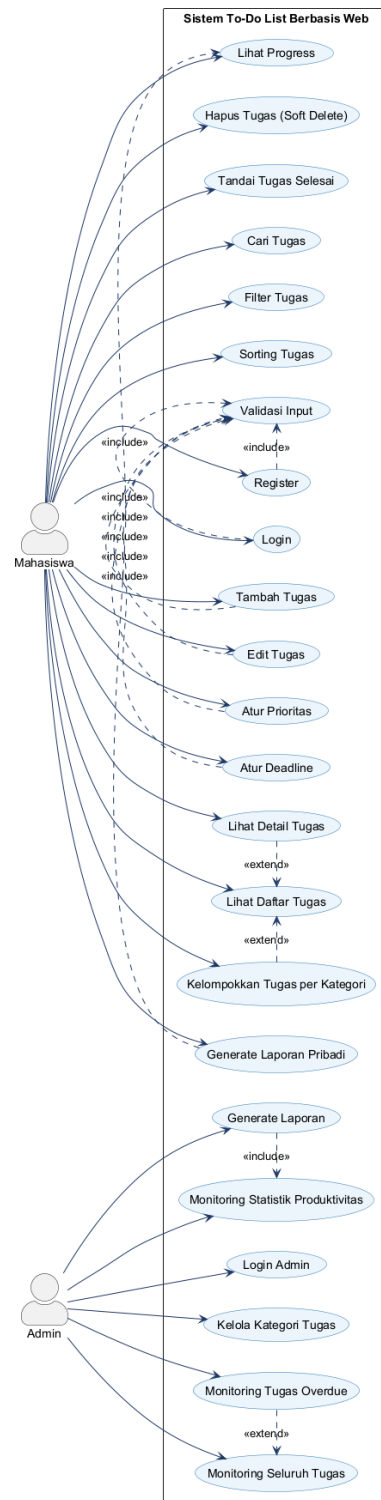
2. Pengguna memahami penggunaan dasar aplikasi berbasis checklist untuk pengelolaan tugas.
3. Data yang dimasukkan oleh pengguna dianggap valid dan digunakan untuk kebutuhan pribadi masing-masing pengguna.
4. Basis data mampu menyimpan data pengguna, kategori, dan tugas secara terpusat tanpa konflik data.
5. Setiap pengguna dapat memiliki kategori tugas yang berbeda tanpa memengaruhi kategori milik pengguna lain.
6. Pengguna diasumsikan melakukan pengelolaan tugas secara mandiri tanpa keterlibatan pengguna lain dalam satu akun.
7. Sistem diasumsikan mampu menangani perubahan data secara real-time pada tampilan antarmuka setelah pengguna melakukan aksi tertentu, seperti menambah, mengedit, atau menandai tugas selesai.

BAB III

MODEL SISTEM

3.1 Use Case Diagram (WAJIB)

a. Diagram Use Case



Gambar 3. 1 Diagram Use Case

b. Identifikasi Aktor

Tabel 3. 1 Tabel Aktor

Aktor	Deskripsi Peran	Hak Akses Utama
Mahasiswa	Pengguna utama sistem yang berperan sebagai pemilik tugas pribadi. Mahasiswa wajib melakukan registrasi dan login sebelum mengakses fitur utama. Mahasiswa berinteraksi langsung dengan seluruh fitur manajemen tugas, mulai dari pencatatan hingga pemantauan progres penyelesaian.	Registrasi akun, login ke sistem, menambah/mengedit/menghapus tugas (soft delete), menandai tugas selesai via checkbox, mencari dan memfilter tugas, mengurutkan tugas, menetapkan prioritas dan deadline, melihat daftar tugas, melihat detail tugas, mengelompokkan tugas per kategori, memantau progress penyelesaian tugas pribadi, serta generate laporan tugas pribadi.
Admin	Pengelola sistem yang bertanggung jawab atas monitoring agregat seluruh aktivitas tugas di platform. Admin tidak memiliki tugas pribadi, fokus pada pengelolaan data sistem dan pembuatan laporan.	Memonitor seluruh tugas lintas pengguna, memantau tugas overdue, menganalisis statistik produktivitas, mengelola master data kategori tugas, dan menghasilkan laporan periodik sistem.

1. Relasi (include, extend, generalization)

Tabel 3. 2 Tabel Relasi

Relasi	Asal Use Case	Target Use Case	Jenis	Alasan
R00	Register	Validasi Input	<<include>>	Proses pendaftaran akun baru wajib melewati validasi input seperti format email, kekuatan password, dan keunikan email sebelum data disimpan.
R01	Login	Validasi Input	<<include>>	Proses login memerlukan validasi input untuk memastikan field email dan password tidak kosong serta format email sesuai sebelum dikirim ke server.
R02	Tambah Tugas	Validasi Input	<<include>>	Setiap penambahan tugas wajib melewati validasi input (field kosong, format tanggal, dll). Proses validasi selalu dieksekusi, bukan opsional.

R03	Edit Tugas	Validasi Input	<<include>>	Perubahan data tugas juga memerlukan validasi yang sama agar data tetap konsisten di database.
R04	Atur Prioritas	Validasi Input	<<include>>	Penetapan prioritas merupakan bagian dari form tugas; input nilainya (High/Medium/Low) perlu divalidasi.
R05	Atur Deadline	Validasi Input	<<include>>	Tanggal deadline harus divalidasi formatnya dan tidak boleh bernilai tanggal lampau saat pertama kali ditambahkan.
R06	Monitoring Tugas Overdue	Monitoring Seluruh Tugas	<<extend>>	Monitoring overdue adalah tampilan terfilter dari data monitoring seluruh tugas. Bersifat kondisional (opsional), hanya ditampilkan ketika admin memilih filter overdue.
R07	Generate Laporan	Monitoring Statistik Produktivitas	<<include>>	Laporan yang dihasilkan selalu menyertakan data statistik produktivitas sebagai konten utamanya. Proses generate laporan tidak dapat berjalan tanpa data statistik.
R08	Lihat Detail Tugas	Lihat Daftar Tugas	<<extend>>	Lihat detail tugas merupakan perluasan opsional dari daftar tugas. Mahasiswa hanya membuka panel detail jika ingin melihat informasi lengkap tugas tertentu.
R09	Kelompokkan Tugas per Kategori	Lihat Daftar Tugas	<<extend>>	Pengelompokan per kategori adalah tampilan alternatif opsional dari daftar tugas. Aktif hanya saat mahasiswa memilih mode tampilan per kategori.
R10	Generate Laporan Pribadi	Lihat Progress	<<include>>	Laporan pribadi mahasiswa selalu menyertakan data progress penyelesaian tugas sebagai konten utamanya. Proses generate tidak dapat berjalan tanpa data progress.

2. Deskripsi singkat tiap Use Case

Tabel 3. 3 Tabel Deskripsi Diagram Use Case

Kode UC	Nama Use Case	Aktor	Definisi Use Case	Deskripsi Singkat
UC00	Register	Mahasiswa	Proses pembuatan akun baru oleh calon pengguna agar dapat terdaftar dan	Mahasiswa mengisi form registrasi berisi nama lengkap, email, dan password. Sistem memvalidasi keunikan email dan

			mengakses sistem DoneRight.	kekuatan password, lalu menyimpan akun baru dengan role mahasiswa.
UC01	Login	Mahasiswa	Proses autentikasi yang memverifikasi identitas pengguna agar dapat mengakses sistem secara aman dan personal.	Mahasiswa memasukkan kredensial (email/username dan password) untuk mengakses dashboard tugas pribadi. Sistem memverifikasi identitas dan membuat sesi aktif.
UC02	Tambah Tugas	Mahasiswa	Proses pencatatan tugas baru ke dalam sistem dengan mengisi atribut-atribut yang diperlukan untuk pengelolaan tugas.	Mahasiswa mengisi form penambahan tugas baru yang mencakup judul, deskripsi, kategori, prioritas, dan deadline. Sistem menyimpan tugas dengan status Aktif.
UC03	Validasi Input	Sistem	Proses pemeriksaan otomatis oleh sistem terhadap data yang dimasukkan pengguna untuk memastikan kelengkapan, format, dan kevalidan nilai sebelum data disimpan.	Sistem memeriksa kelengkapan dan kevalidan data form tugas, meliputi field wajib tidak boleh kosong, format tanggal deadline, serta nilai enum prioritas dan kategori yang diizinkan.
UC04	Lihat Daftar Tugas	Mahasiswa	Proses menampilkan keseluruhan data tugas milik mahasiswa dalam bentuk daftar yang informatif dan mudah dibaca.	Mahasiswa melihat seluruh tugas pribadi dalam tampilan daftar yang dapat diurutkan dan difilter. Daftar menampilkan status, prioritas, dan sisa waktu deadline tiap tugas.
UC05	Edit Tugas	Mahasiswa	Proses pembaruan data tugas yang sudah ada untuk menyesuaikan informasi tugas dengan kondisi atau kebutuhan terkini.	Mahasiswa mengubah satu atau lebih atribut tugas yang sudah ada (judul, deskripsi, kategori, prioritas, atau deadline). Perubahan disimpan ke database setelah melewati validasi.
UC06	Hapus Tugas (Soft Delete)	Mahasiswa	Proses penghapusan tugas menggunakan mekanisme soft delete, di mana data tidak dihapus secara fisik dari database melainkan ditandai sebagai terhapus.	Mahasiswa menghapus tugas dari tampilan daftar aktif. Sistem menampilkan dialog konfirmasi, lalu menandai tugas dengan flag <code>deleted_at</code> sehingga tidak muncul di daftar tanpa menghapus data dari database.

UC07	Tandai Tugas Selesai	Mahasiswa	Proses perubahan status tugas menjadi selesai melalui mekanisme checkbox sebagai penanda bahwa pengerjaan tugas telah berhasil diselesaikan oleh mahasiswa.	Mahasiswa mencentang checkbox pada tugas untuk mengubah statusnya dari Aktif menjadi Selesai. Sistem mencatat timestamp penyelesaian dan memperbarui persentase progress keseluruhan secara otomatis.
UC08	Cari Tugas	Mahasiswa	Proses pencarian tugas secara cepat menggunakan kata kunci untuk menemukan tugas tertentu di antara banyak tugas yang tersimpan.	Mahasiswa mencari tugas menggunakan kata kunci yang dicocokkan dengan judul atau deskripsi tugas. Hasil pencarian ditampilkan secara real-time pada daftar.
UC09	Filter Tugas	Mahasiswa	Proses penyaringan tampilan daftar tugas berdasarkan kriteria tertentu agar mahasiswa hanya melihat tugas yang relevan dengan kebutuhannya.	Mahasiswa memfilter tampilan daftar tugas berdasarkan satu atau kombinasi dari: status (Aktif/Selesai/Overdue), prioritas (High/Medium/Low), atau kategori tugas.
UC10	Sorting Tugas	Mahasiswa	Proses pengurutan tampilan daftar tugas berdasarkan kolom tertentu untuk memudahkan mahasiswa menentukan tugas mana yang harus dikerjakan lebih dahulu.	Mahasiswa mengurutkan daftar tugas berdasarkan kolom deadline (ascending/descending) atau prioritas (High ke Low atau sebaliknya) sesuai kebutuhan.
UC11	Atur Prioritas	Mahasiswa	Proses penetapan atau perubahan tingkat kepentingan suatu tugas relatif terhadap tugas-tugas lainnya untuk membantu manajemen waktu pengerjaan.	Mahasiswa menetapkan atau mengubah tingkat prioritas tugas menjadi High, Medium, atau Low. Nilai prioritas divalidasi dan digunakan sebagai dasar sorting dan filter.
UC12	Atur Deadline	Mahasiswa	Proses penetapan atau perubahan batas waktu pengerjaan tugas agar	Mahasiswa menetapkan atau mengubah tanggal batas pengerjaan tugas. Sistem akan

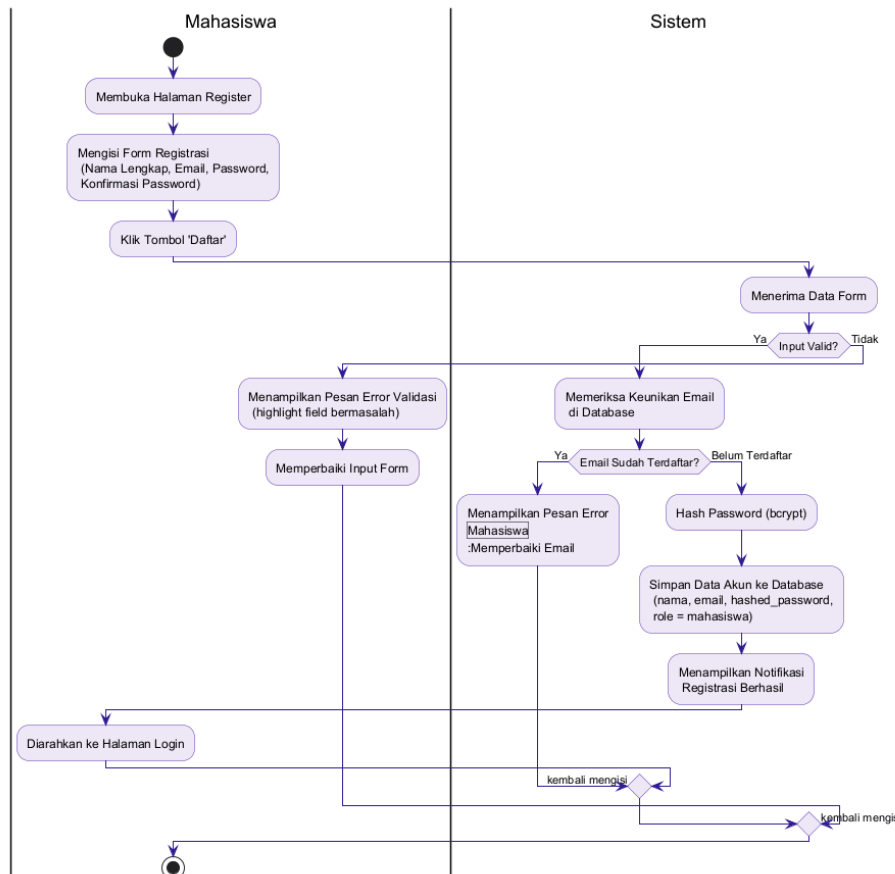
			sistem dapat memantau ketepatan waktu penyelesaian secara otomatis.	otomatis mengubah status tugas menjadi Overdue apabila deadline terlampaui dan tugas belum selesai.
UC13	Lihat Progress	Mahasiswa	Proses pemantauan tingkat penyelesaian tugas secara keseluruhan dalam bentuk visualisasi persentase untuk memberikan gambaran produktivitas mahasiswa.	Mahasiswa memantau persentase penyelesaian tugas pribadi (jumlah tugas selesai dibanding total tugas aktif). Ditampilkan dalam bentuk widget progress bar pada halaman dashboard.
UC14	Login Admin	Admin	Proses autentikasi khusus yang memverifikasi identitas admin dan memberikan hak akses penuh ke panel administrasi sistem.	Admin mengakses panel administrasi menggunakan kredensial khusus admin. Sesi admin memiliki hak akses dan tampilan yang berbeda dari sesi mahasiswa biasa.
UC15	Monitoring Seluruh Tugas	Admin	Proses pemantauan dan peninjauan agregat seluruh data tugas dari semua mahasiswa terdaftar di platform untuk keperluan pengawasan sistem.	Admin melihat keseluruhan data tugas lintas mahasiswa dalam bentuk tabel yang dapat difilter berdasarkan status, prioritas, kategori, atau rentang waktu tertentu.
UC16	Monitoring Tugas Overdue	Admin	Proses pemantauan khusus terhadap tugas-tugas yang telah melewati batas deadline tanpa diselesaikan, sebagai indikator produktivitas dan kepatuhan mahasiswa.	Admin melihat daftar tugas berstatus Overdue dari seluruh mahasiswa, lengkap dengan informasi durasi keterlambatan dan identitas mahasiswa pemilik tugas.
UC17	Monitoring Statistik Produktivitas	Admin	Proses pengumpulan dan penampilan data statistik agregat yang merepresentasikan tingkat produktivitas penggunaan platform secara keseluruhan.	Admin melihat ringkasan statistik produktivitas platform yang meliputi total tugas, tingkat penyelesaian, distribusi prioritas, dan tren penyelesaian per periode waktu.

UC18	Kelola Kategori Tugas	Admin	Proses pengelolaan master data kategori tugas oleh admin sebagai data referensi yang digunakan seluruh mahasiswa saat membuat atau mengedit tugas.	Admin menambah atau menghapus master data kategori tugas yang berlaku secara global untuk seluruh pengguna sistem. Mahasiswa tidak dapat membuat kategori sendiri, melainkan memilih dari kategori yang telah disediakan admin.
UC19	Generate Laporan	Admin	Proses pembuatan dokumen laporan ringkasan sistem yang dapat diunduh, berisi data statistik dan daftar tugas sesuai parameter yang dipilih admin.	Admin menghasilkan laporan dalam format yang dapat diunduh (PDF/CSV), berisi data statistik produktivitas dan daftar tugas berdasarkan filter status, periode, atau kategori.
UC20	Lihat Detail Tugas	Mahasiswa	Proses menampilkan informasi lengkap sebuah tugas melalui mekanisme dropdown atau panel detail tanpa berpindah halaman.	Mahasiswa mengklik sebuah tugas dari daftar untuk memperluas panel detail yang menampilkan seluruh atribut tugas secara lengkap termasuk deskripsi, kategori, prioritas, deadline, dan status.
UC21	Kelompokkan Tugas per Kategori	Mahasiswa	Proses menampilkan daftar tugas yang dikelompokkan berdasarkan kategorinya sehingga mahasiswa dapat melihat tugas secara terorganisir per kelompok.	Mahasiswa melihat daftar tugas yang disusun dalam kelompok-kelompok berdasarkan kategori (Akademik, Organisasi, Pribadi, atau kategori kustom). Setiap kelompok menampilkan jumlah tugas di dalamnya.
UC22	Generate Laporan Pribadi	Mahasiswa	Proses pembuatan laporan ringkasan tugas pribadi oleh mahasiswa yang dapat diunduh, berisi data progress dan daftar tugas sesuai filter yang dipilih.	Mahasiswa menghasilkan laporan tugas pribadi dalam format yang dapat diunduh (PDF/CSV), berisi data progress penyelesaian dan daftar tugas berdasarkan filter status, prioritas, atau rentang waktu tertentu. Laporan hanya mencakup data tugas milik mahasiswa yang bersangkutan.

3.2 Activity Diagram

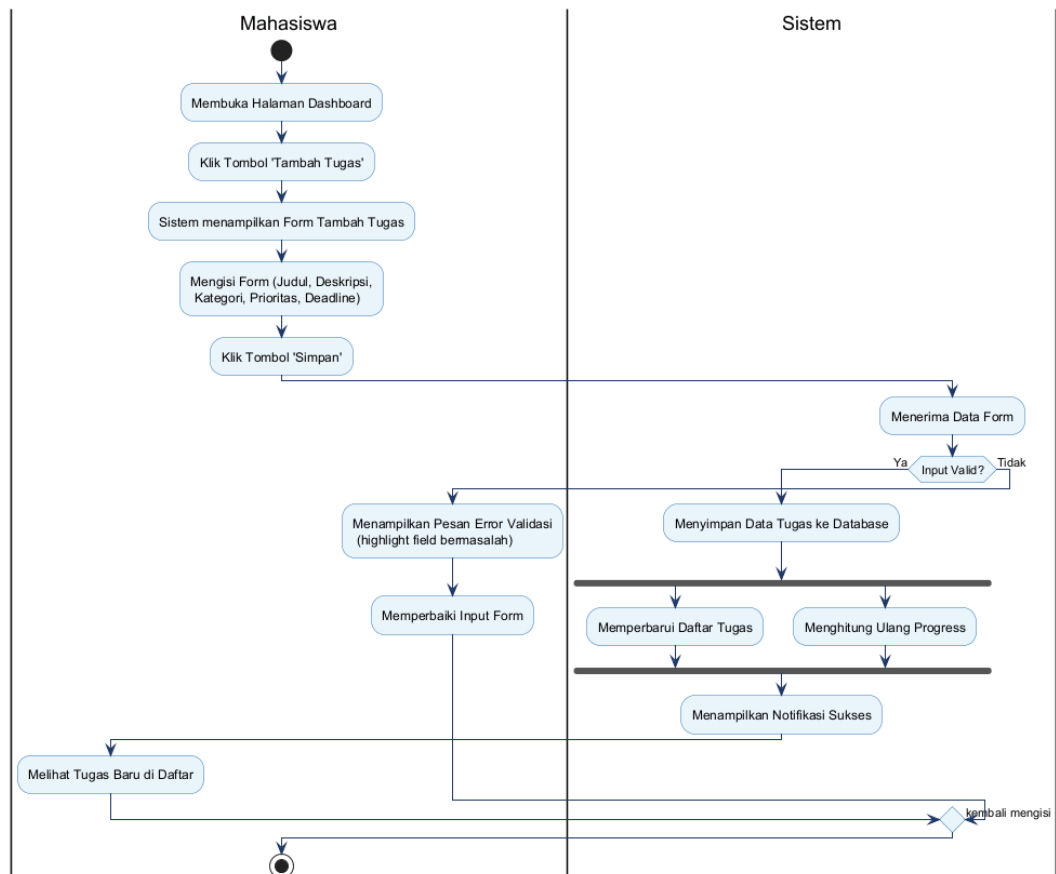
a. Alur proses utama sistem

3. Minimal 1–2 proses inti
4. Menunjukkan decision, parallel flow (jika ada)

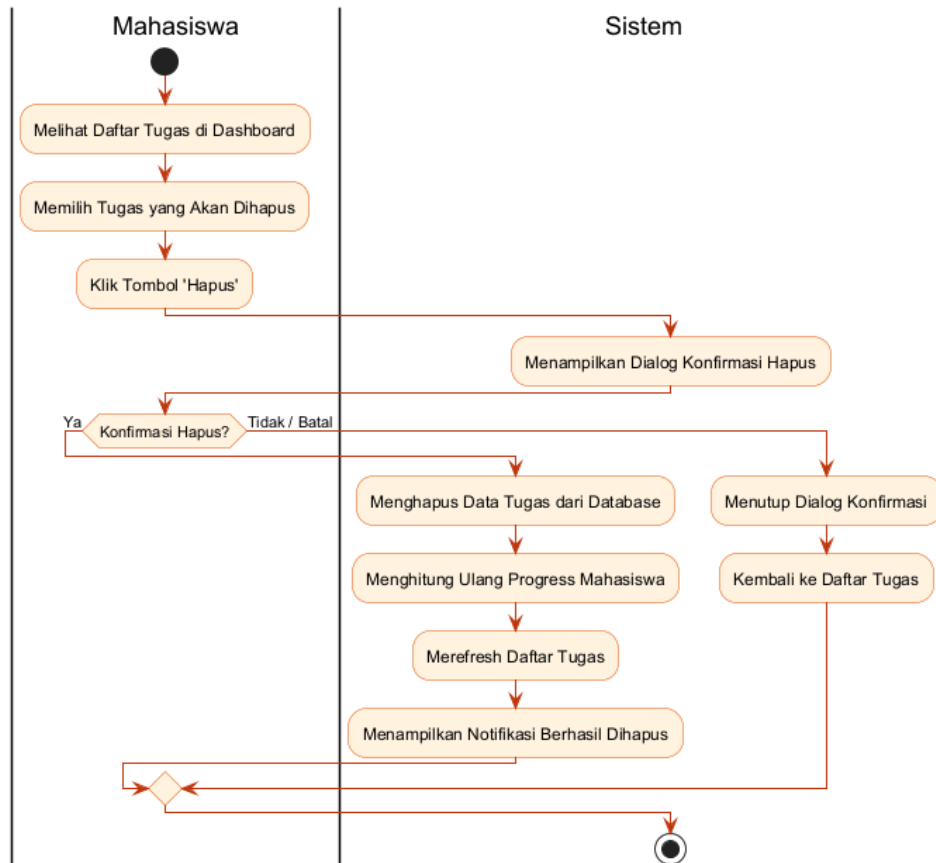


Gambar 3. 2 Activity Diagram Registrasi

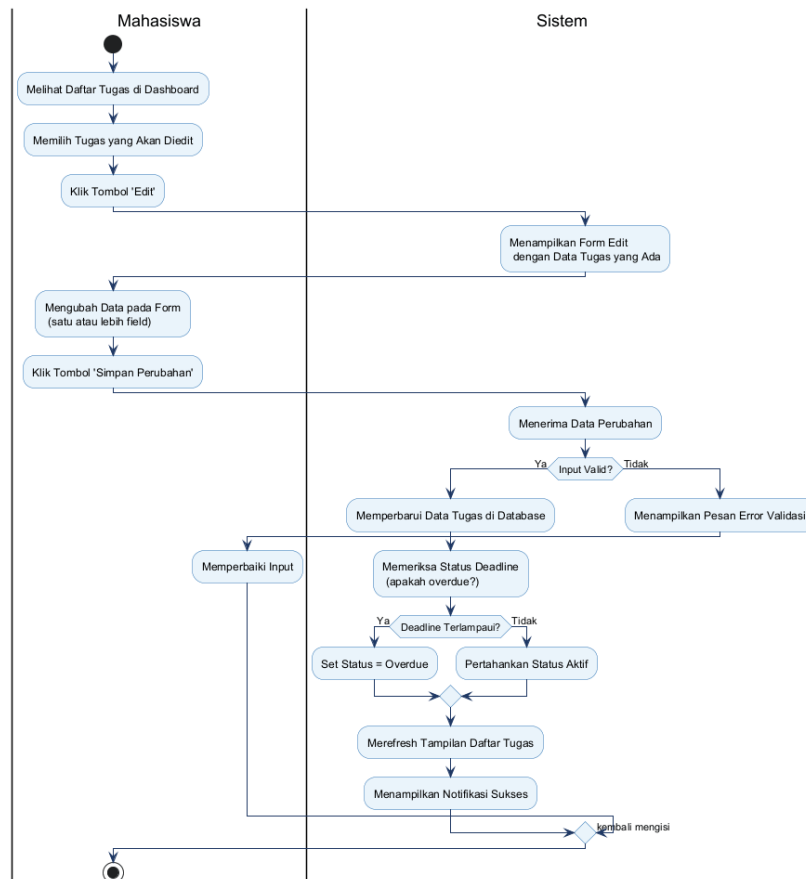
Proses dimulai ketika mahasiswa membuka halaman Register dan mengisi form registrasi berisi nama lengkap, email, password, dan konfirmasi password, lalu mengklik tombol "Daftar". Sistem menerima data form dan melakukan validasi input; jika tidak valid, pesan error ditampilkan dan mahasiswa diminta memperbaiki input form. Jika input valid, sistem memeriksa keunikan email di database, apabila email sudah terdaftar, sistem menampilkan pesan error dan mahasiswa diminta memperbaiki email. Jika email belum terdaftar, sistem melakukan hash password menggunakan bcrypt, menyimpan data akun baru ke database dengan role mahasiswa, lalu menampilkan notifikasi registrasi berhasil. Mahasiswa kemudian diarahkan ke halaman Login untuk melanjutkan proses autentikasi.



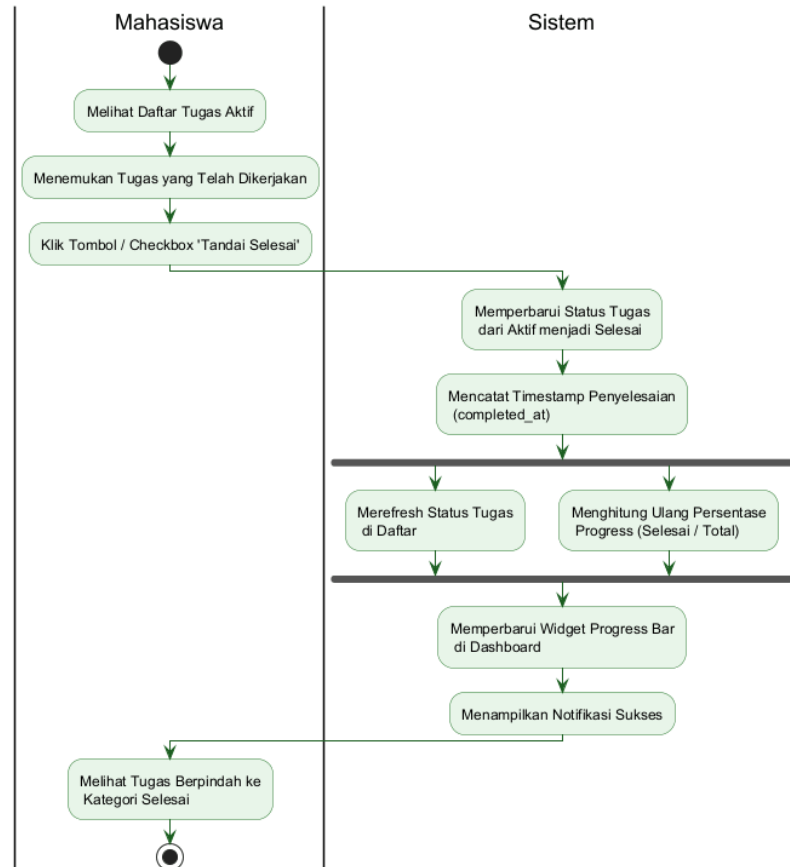
Alur dimulai ketika mahasiswa membuka halaman dashboard dan mengklik tombol "Tambah Tugas". Sistem menampilkan form yang kemudian diisi oleh mahasiswa dengan data judul, deskripsi, kategori, prioritas, dan deadline, lalu dikonfirmasi dengan klik "Simpan". Sistem kemudian memvalidasi input; jika tidak valid, pesan error ditampilkan dan mahasiswa diminta memperbaiki input. Jika valid, sistem menyimpan data ke database, memperbarui daftar tugas dan menghitung ulang progress secara paralel, kemudian menampilkan notifikasi sukses kepada mahasiswa.



Alur dimulai ketika mahasiswa memilih tugas yang ingin dihapus dari daftar dan mengklik tombol "Hapus". Sistem menampilkan dialog konfirmasi untuk mencegah penghapusan tidak disengaja. Jika mahasiswa membatalkan, dialog ditutup dan tampilan kembali ke daftar tugas. Jika mahasiswa mengkonfirmasi, sistem menghapus data tugas dari database, menghitung ulang progress, merefresh daftar tugas, lalu menampilkan notifikasi keberhasilan penghapusan.



Alur dimulai ketika mahasiswa memilih tugas yang akan diubah dan mengklik tombol "Edit". Sistem menampilkan form edit yang sudah terisi data tugas saat ini. Mahasiswa mengubah satu atau lebih field lalu mengklik "Simpan Perubahan". Sistem memvalidasi input; jika tidak valid, pesan error ditampilkan dan mahasiswa diminta memperbaiki. Jika valid, sistem memperbarui data di database, memeriksa apakah deadline sudah terlampaui untuk menentukan status tugas (Overdue atau tetap Aktif), kemudian meresh daftar tugas dan menampilkan notifikasi sukses.



Alur dimulai ketika mahasiswa menemukan tugas yang telah dikerjakan pada daftar tugas aktif dan mengklik tombol atau checkbox "Tandai Selesai". Sistem langsung memperbarui status tugas dari Aktif menjadi Selesai dan mencatat timestamp penyelesaian. Secara paralel, sistem merefresh status tugas di daftar sekaligus menghitung ulang persentase progress, kemudian memperbarui widget progress bar di dashboard dan menampilkan notifikasi sukses. Mahasiswa dapat melihat tugas tersebut telah berpindah ke kategori Selesai.

3.3 Sequence Diagram

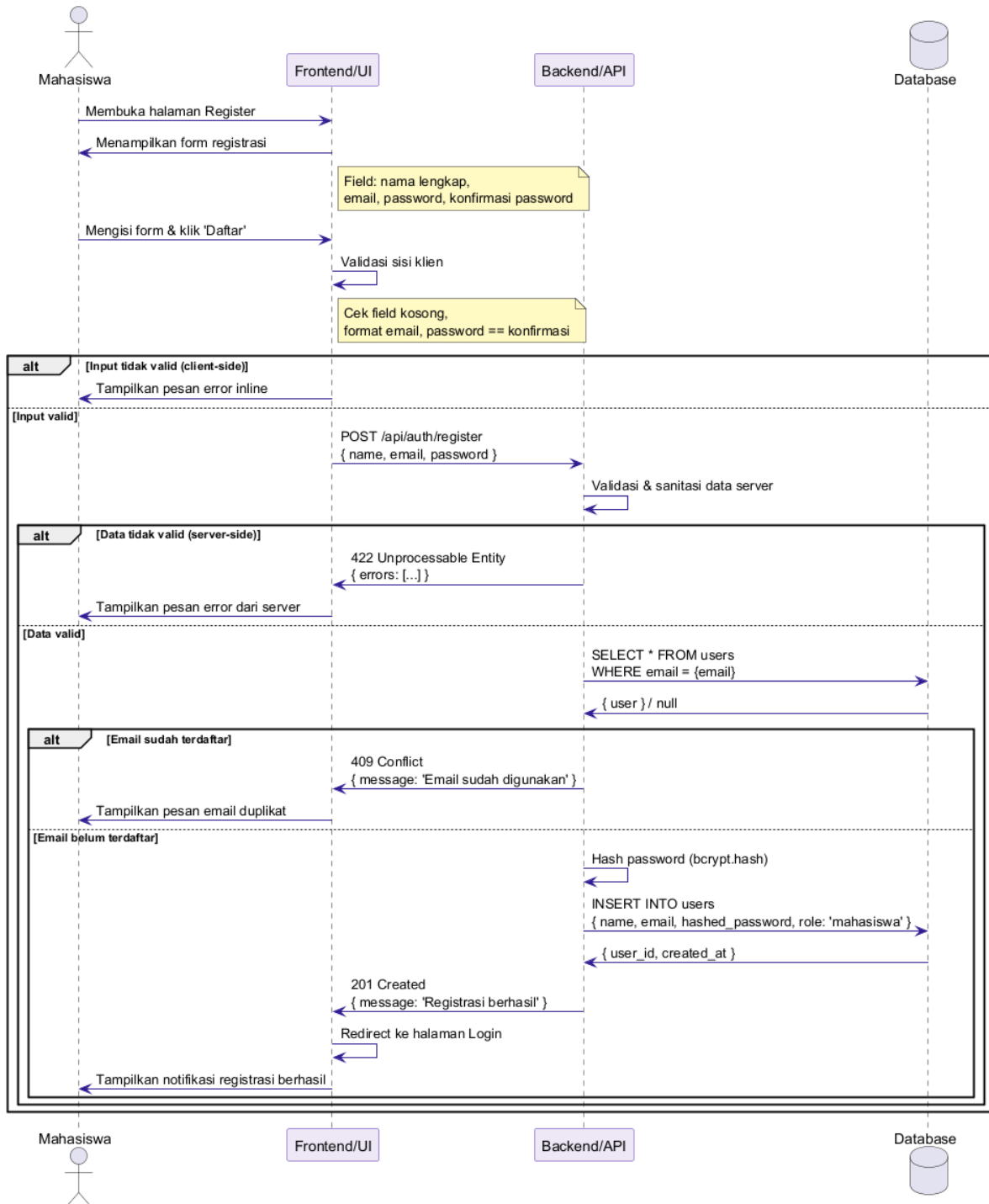
a. Interaksi antar komponen:

b. User – Frontend – Backend – Database

c. Fokus pada skenario utama (contoh: login, transaksi, dll)

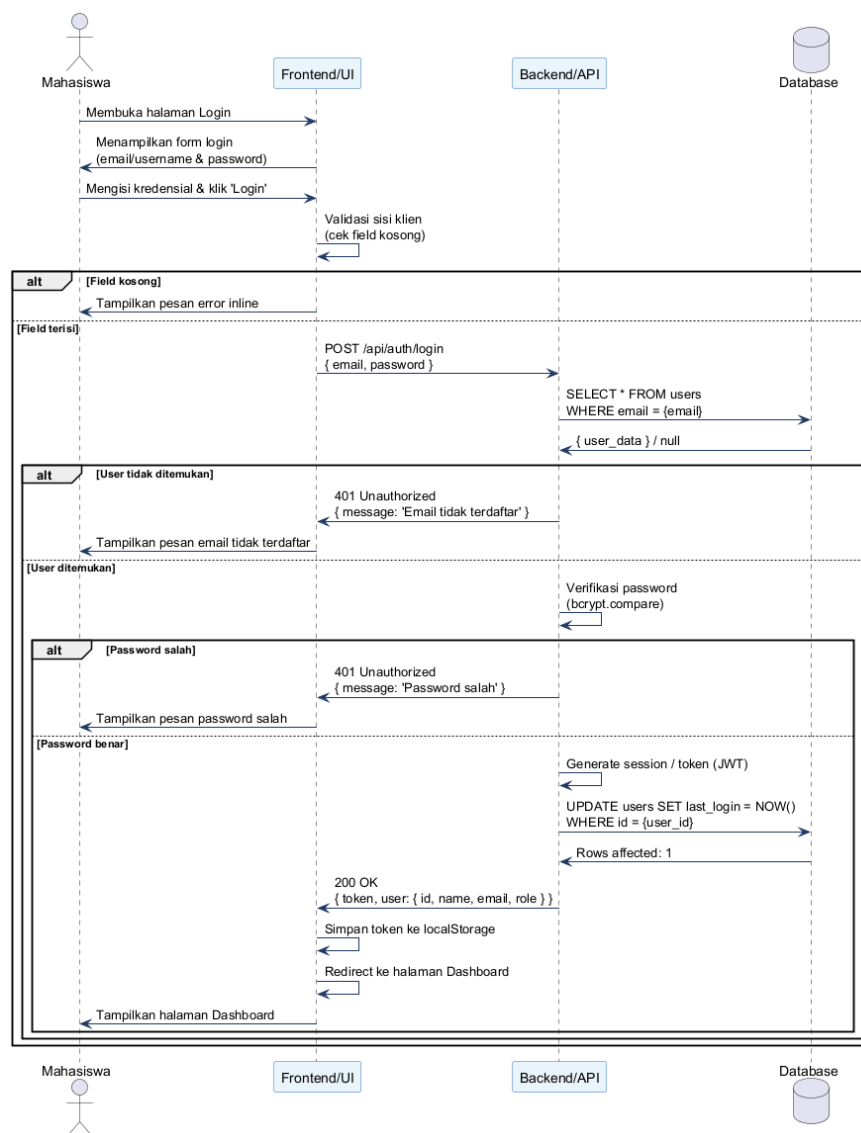
Nama Atribut	Tipe Data	Keterangan
user_id	UUID	Primary key pengguna
full_name	String	Nama lengkap pengguna
email	String	Email unik pengguna

password_hash	String	Password yang sudah di-hash
role	String	Role pengguna (mahasiswa/admin)
created_at	Timestamp	Waktu akun dibuat
updated_at	Timestamp	Waktu data diperbarui
last_login	Timestamp	Waktu login terakhir



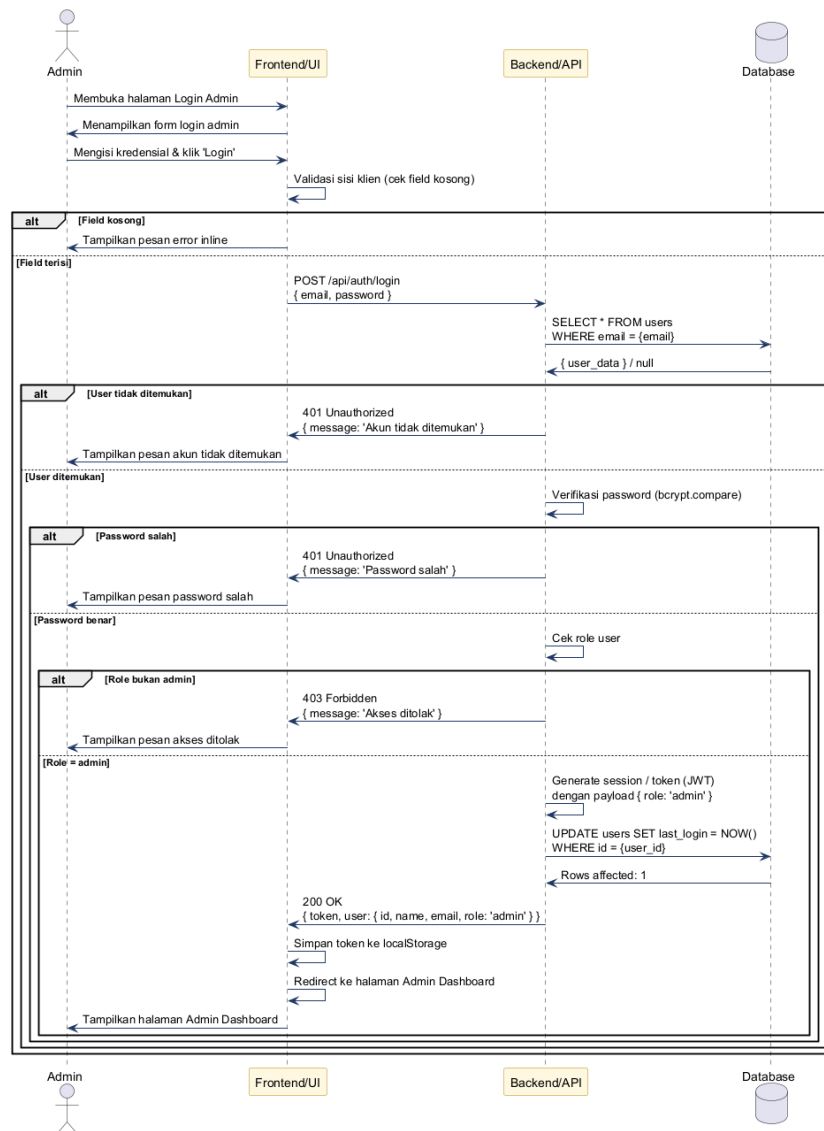
Proses dimulai ketika Mahasiswa mengisi form registrasi dengan *field* nama lengkap, email, *password*, dan konfirmasi *password*, lalu mengklik tombol "Daftar". *Frontend*

melakukan validasi sisi klien terlebih dahulu untuk mengecek field kosong, format email, serta kesesuaian antara *password* dan konfirmasi *password*. Jika ditemukan input tidak valid, pesan error ditampilkan secara inline tanpa mengirim request ke server. Jika lolos validasi klien, *Frontend* mengirim `POST /api/auth/register` beserta data pendaftaran ke Backend. *Backend* kemudian melakukan validasi dan sanitasi ulang di sisi server. Jika gagal, sistem mengembalikan status `422 Unprocessable Entity`. Jika data valid, *Backend* melakukan pengecekan ke Database untuk memastikan email belum terdaftar; jika email sudah ada, *Backend* mengembalikan status `409 Conflict`. Namun, jika email tersedia, Backend akan melakukan hashing password menggunakan `bcrypt`, mengeksekusi `INSERT INTO users` dengan default role 'mahasiswa', dan mengembalikan respons `201 Created`. Sebagai tahap akhir, Frontend akan melakukan redirect ke halaman Login serta menampilkan notifikasi keberhasilan registrasi kepada Mahasiswa.



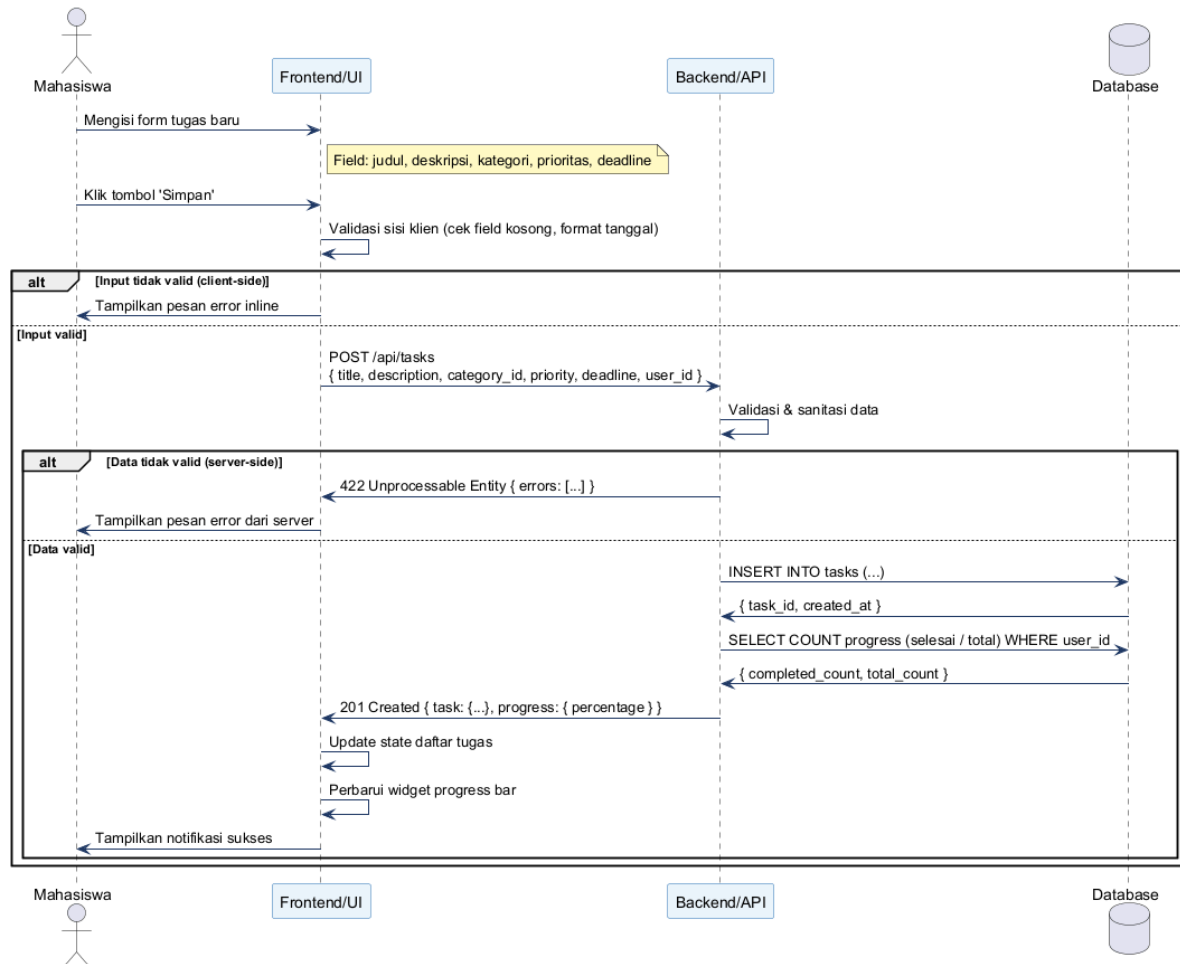
Proses dimulai ketika Mahasiswa membuka halaman login dan mengisi form email/username beserta *password*. *Frontend* memvalidasi kelengkapan *field*. Jika kosong, error inline langsung

ditampilkan. Jika terisi, Frontend mengirim `POST /api/auth/login` ke Backend. Backend mencari data user di database berdasarkan email. Jika tidak ditemukan, mengembalikan `401 Unauthorized`. Jika user ditemukan, *Backend* memverifikasi *password* dengan *bcrypt*; jika salah, kembali mengembalikan `401 Unauthorized`. Jika password benar, *Backend* membuat token JWT, memperbarui kolom `last_login` di *database*, lalu mengembalikan `200 OK` dengan data user dan token. Frontend menyimpan token dan mengarahkan Mahasiswa ke halaman *Dashboard*.

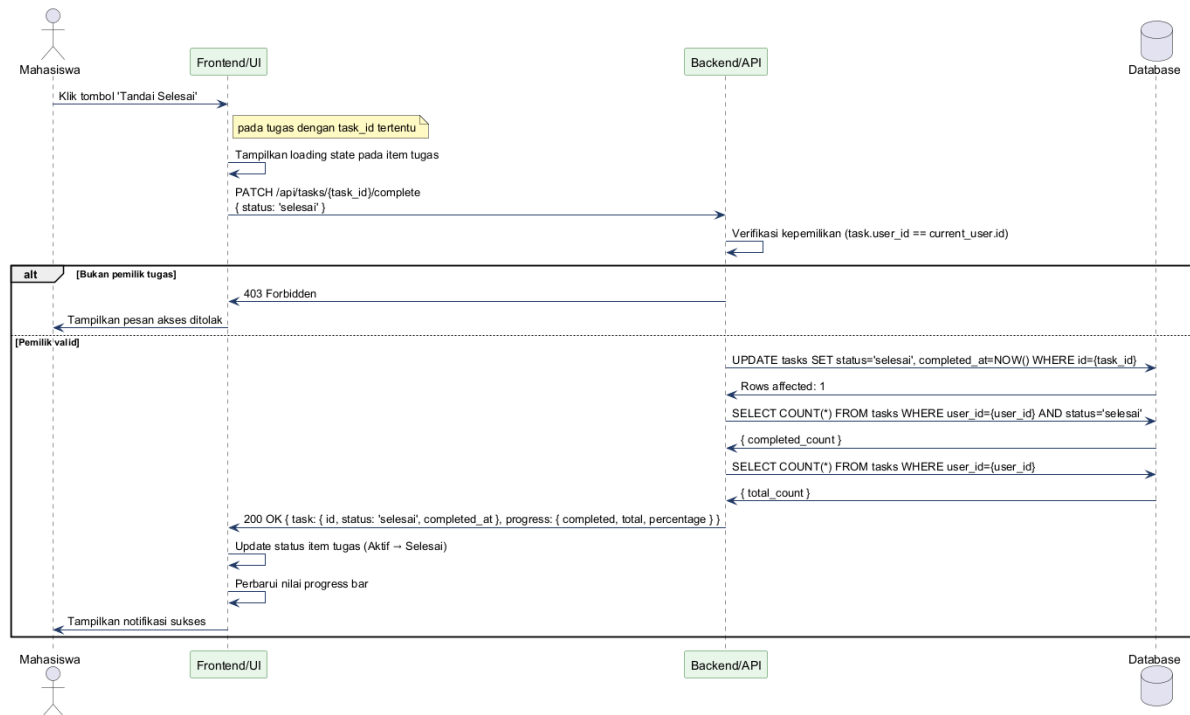


Proses dimulai ketika Admin membuka halaman login dan mengisi form kredensial. *Frontend* memvalidasi kelengkapan *field* secara lokal; jika kosong, error langsung ditampilkan tanpa meneruskan request. Jika field terisi, Frontend mengirim request `POST /api/auth/login` ke Backend. Backend mengecek keberadaan akun di *database*, lalu memverifikasi password menggunakan *bcrypt*. Setelah password terkonfirmasi benar, Backend melakukan pengecekan

tambahan pada role user. Jika bukan admin, request ditolak dengan status 403 `Forbidden`. Jika role valid sebagai admin, Backend membuat token JWT dengan payload role admin, mencatat waktu login terakhir ke database, dan mengembalikan response 200 `OK` beserta data user. *Frontend* menyimpan token dan mengarahkan Admin ke halaman Admin Dashboard.



Proses dimulai ketika Mahasiswa mengisi form tambah tugas dengan *field* judul, deskripsi, kategori, prioritas, dan *deadline*, lalu mengklik tombol "Simpan". *Frontend* melakukan validasi sisi klien terlebih dahulu, jika ada *field* kosong atau format tanggal tidak sesuai, error ditampilkan langsung tanpa mengirim request. Jika lolos validasi klien, *Frontend* mengirim `POST /api/tasks` beserta seluruh data *form* ke *Backend*. Backend melakukan validasi dan sanitasi ulang di sisi server. Jika gagal, mengembalikan `422 Unprocessable Entity`. Jika data valid, *Backend* mengeksekusi `INSERT INTO tasks` ke *database*, kemudian mengambil data progress terbaru (jumlah tugas selesai dan total). Backend mengembalikan `201 Created` dengan data tugas baru dan persentase progress. Frontend memperbarui state daftar tugas, mererefresh *widget progress bar*, dan menampilkan notifikasi sukses kepada Mahasiswa.



Proses dimulai ketika Mahasiswa mengklik tombol "Tandai Selesai" pada tugas tertentu. Frontend menampilkan loading state pada item tugas tersebut, lalu mengirim `PATCH /api/tasks/{task_id}/complete` ke Backend. Backend memverifikasi kepemilikan tugas dengan mencocokkan `task.user_id` dan `current_user.id`. Jika tidak cocok, request ditolak dengan `403 Forbidden`. Jika kepemilikan valid, Backend mengeksekusi `UPDATE` untuk mengubah status tugas menjadi `selesai` dan mencatat `completed_at`. Selanjutnya Backend menghitung ulang progress dengan dua query terpisah: jumlah tugas selesai dan total tugas milik user. Backend mengembalikan `200 OK` dengan data tugas yang diperbarui beserta nilai progress terbaru. Frontend memperbarui tampilan status tugas dari Aktif menjadi Selesai, mererefresh nilai progress bar, dan menampilkan notifikasi sukses.

1.5. Class Diagram

3.4.1 Gambaran Umum Class Diagram

Class Diagram pada sistem DoneRight digunakan untuk menggambarkan struktur utama sistem, hubungan antar class, atribut yang dimiliki setiap class, serta method yang digunakan dalam proses bisnis aplikasi. Diagram ini dirancang berdasarkan kebutuhan fungsional yang telah dijelaskan pada dokumen SRS dan disesuaikan dengan implementasi aplikasi berbasis web menggunakan React.js, Node.js, Express.js, dan Supabase PostgreSQL.

Class Diagram pada sistem DoneRight terdiri atas beberapa class utama, yaitu:

1. User

2. Admin
3. Task
4. Category
5. Report
6. AuthService
7. TaskService
8. CategoryService
9. ReportService

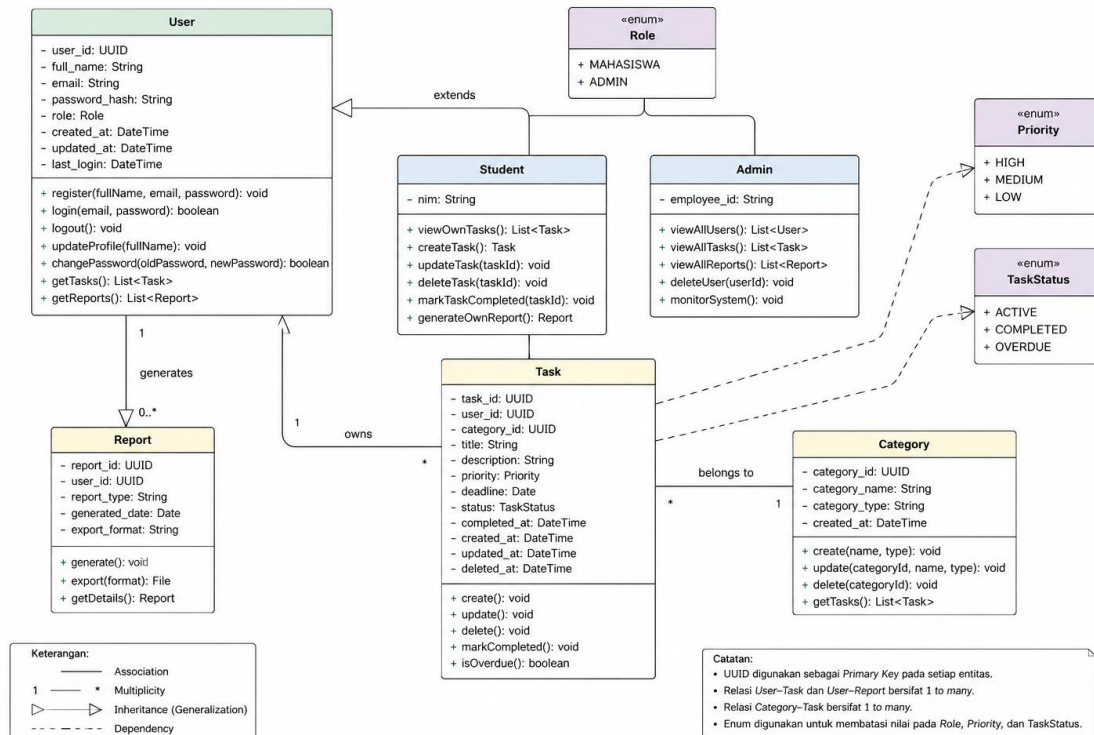
Relasi antar class menggunakan konsep association, inheritance, dan aggregation agar struktur sistem lebih terorganisasi serta mudah diimplementasikan ke dalam basis data maupun kode program.

3.4.2 Daftar Class Utama

Class utama pada sistem DoneRight terdiri atas beberapa class inti yang merepresentasikan data utama sistem dan service pendukung proses bisnis aplikasi.

No	Nama Class	Fungsi Utama
1	User	Menyimpan data dan aktivitas pengguna sistem
2	Admin	Mengelola monitoring dan administrasi sistem
3	Task	Mengelola data tugas pengguna
4	Category	Mengelola kategori tugas
5	Report	Mengelola generate laporan
6	AuthService	Menangani autentikasi dan keamanan
7	TaskService	Menangani logika bisnis tugas
8	CategoryService	Menangani logika kategori
9	ReportService	Menangani generate statistik dan laporan

3.4.3 Diagram Class



3.4.4 Atribut dan Method Class

1. Class User

Class User merepresentasikan pengguna utama sistem DoneRight yang memiliki akun untuk mengakses aplikasi. Class ini menyimpan data identitas pengguna dan menangani aktivitas autentikasi dasar.

Atribut User

Nama Atribut	Tipe Data	Keterangan
user_id	UUID	Primary key pengguna
full_name	String	Nama lengkap pengguna
email	String	Email unik pengguna
password_hash	String	Password yang sudah di-hash
role	String	Role pengguna (mahasiswa/admin)
created_at	Timestamp	Waktu akun dibuat
updated_at	Timestamp	Waktu data diperbarui
last_login	Timestamp	Waktu login terakhir

Method User

Method	Fungsi
register()	Melakukan registrasi akun
login()	Melakukan autentikasi pengguna
logout()	Mengakhiri sesi login
viewTasks()	Menampilkan daftar tugas
generatePersonalReport()	Membuat laporan tugas pribadi

2. Class Admin

Class Admin merupakan turunan (inheritance) dari class User yang memiliki hak akses tambahan untuk melakukan monitoring sistem.

Method Admin

Method	Fungsi
monitorTasks()	Melihat seluruh tugas pengguna
monitorOverdueTasks()	Memantau tugas overdue
viewStatistics()	Melihat statistik produktivitas
manageCategory()	Mengelola kategori tugas
generateSystemReport()	Membuat laporan sistem

3. Class Task

Class Task merepresentasikan data tugas yang dikelola oleh pengguna.

Atribut Task

Nama Atribut	Tipe Data	Keterangan
task_id	UUID	Primary key tugas
user_id	UUID	Foreign key ke User
category_id	UUID	Foreign key ke Category
title	String	Judul tugas
description	Text	Deskripsi tugas
priority	Enum	Tingkat prioritas (High/Medium/Low)
deadline	Date	Batas waktu tugas
status	Enum	Status tugas
completed_at	Timestamp	Waktu tugas selesai
created_at	Timestamp	Waktu tugas dibuat
updated_at	Timestamp	Waktu tugas diperbarui
deleted_at	Timestamp	Penanda soft delete

Method Task

Method	Fungsi
createTask()	Menambahkan tugas baru
editTask()	Mengubah data tugas
deleteTask()	Menghapus tugas secara soft delete
markAsComplete()	Menandai tugas selesai
searchTask()	Mencari tugas
filterTask()	Memfilter tugas
sortTask()	Mengurutkan tugas
viewTaskDetail()	Menampilkan detail tugas

4. Class Category

Class Category digunakan untuk menyimpan data kategori tugas.

Atribut Category

Nama Atribut	Tipe Data	Keterangan
category_id	UUID	Primary key kategori
category_name	String	Nama kategori
category_type	String	Jenis kategori
created_at	Timestamp	Waktu kategori dibuat

Method Category

Method	Fungsi
addCategory()	Menambahkan kategori
deleteCategory()	Menghapus kategori
viewCategory()	Menampilkan daftar kategori

5. Class Report

Class Report digunakan untuk menghasilkan laporan sistem maupun laporan pribadi pengguna.

Atribut Report

Nama Atribut	Tipe Data	Keterangan
report_id	UUID	Primary key laporan
user_id	UUID	Foreign key pengguna
report_type	String	Jenis laporan
generated_date	Date	Tanggal generate laporan
export_format	String	Format file laporan

Method Report

Method	Fungsi
generateReport()	Membuat laporan
exportPDF()	Mengekspor laporan PDF
exportCSV()	Mengekspor laporan CSV

6. Class AuthService

Class AuthService merupakan service class yang menangani proses autentikasi dan keamanan sistem.

Method AuthService

Method	Fungsi
validateInput()	Validasi input pengguna
hashPassword()	Mengenkripsi password
verifyPassword()	Verifikasi password
generateJWT()	Membuat token JWT
verifyJWT()	Verifikasi token JWT

7. Class TaskService

Class TaskService digunakan untuk menangani logika bisnis terkait tugas.

Method TaskService

Method	Fungsi
calculateProgress()	Menghitung progress tugas
checkOverdue()	Mengecek tugas overdue
updateTaskStatus()	Mengubah status tugas

8. Class CategoryService

Class CategoryService menangani logika pengelolaan kategori.

Method CategoryService

Method	Fungsi
validateCategory()	Validasi kategori
removeTaskRelation()	Menghapus relasi kategori dari tugas

9. Class ReportService

Class ReportService digunakan untuk menghasilkan statistik dan ringkasan laporan.

Method ReportService

Method	Fungsi
generateStatistics()	Membuat statistik produktivitas
generateTaskSummary()	Membuat ringkasan tugas

3.4.5 Relasi Antar Class (Association, Aggregation, Inheritance)

Relasi antar class pada sistem DoneRight digunakan untuk menggambarkan hubungan antar objek dalam sistem.

Jenis Relasi yang Digunakan

1. Association Association digunakan ketika suatu class berhubungan langsung dengan class lain. Contoh: User memiliki Task dan Report.
2. Inheritance Inheritance digunakan ketika suatu class merupakan turunan dari class lain. Contoh: Admin merupakan turunan dari User.

3. Aggregation Aggregation digunakan ketika suatu class menjadi bagian dari class lain namun masih dapat berdiri sendiri. Contoh: Category mengelompokkan Task, tetapi Task tetap dapat ada tanpa kategori.

Detail Relasi Antar Class

Relasi	Jenis Relasi	Penjelasan
Admin → User	Inheritance	Admin merupakan turunan dari User
User → Task	Association	Satu user dapat memiliki banyak tugas
Category → Task	Association	Satu kategori dapat digunakan banyak tugas
User → Report	Association	User dapat membuat banyak laporan
TaskService → Task	Dependency	Service mengelola proses tugas
AuthService → User	Dependency	Service melakukan autentikasi user
ReportService → Report	Dependency	Service menghasilkan laporan

3.4.6 Mapping ke Database (Keterkaitan dengan ERD)

Mapping class diagram dengan database dilakukan agar setiap class memiliki representasi langsung pada tabel basis data sehingga implementasi backend lebih konsisten.

Class Diagram memiliki keterkaitan langsung dengan Entity Relationship Diagram (ERD). Mapping dilakukan agar struktur class pada sistem konsisten dengan struktur tabel database.

Class	Tabel Database
User	users
Admin	users (role = admin)
Task	tasks
Category	categories
Report	reports

Dengan mapping tersebut, implementasi backend menggunakan Express.js dan Supabase PostgreSQL dapat dilakukan secara lebih konsisten karena setiap class memiliki representasi langsung terhadap tabel basis data.

3.5 Opsional – Nilai Tambah

- a. State Machine Diagram (untuk sistem berbasis status)
- b. Component Diagram (arsitektur modul)
- c. Deployment Diagram (arsitektur fisik)

BAB IV

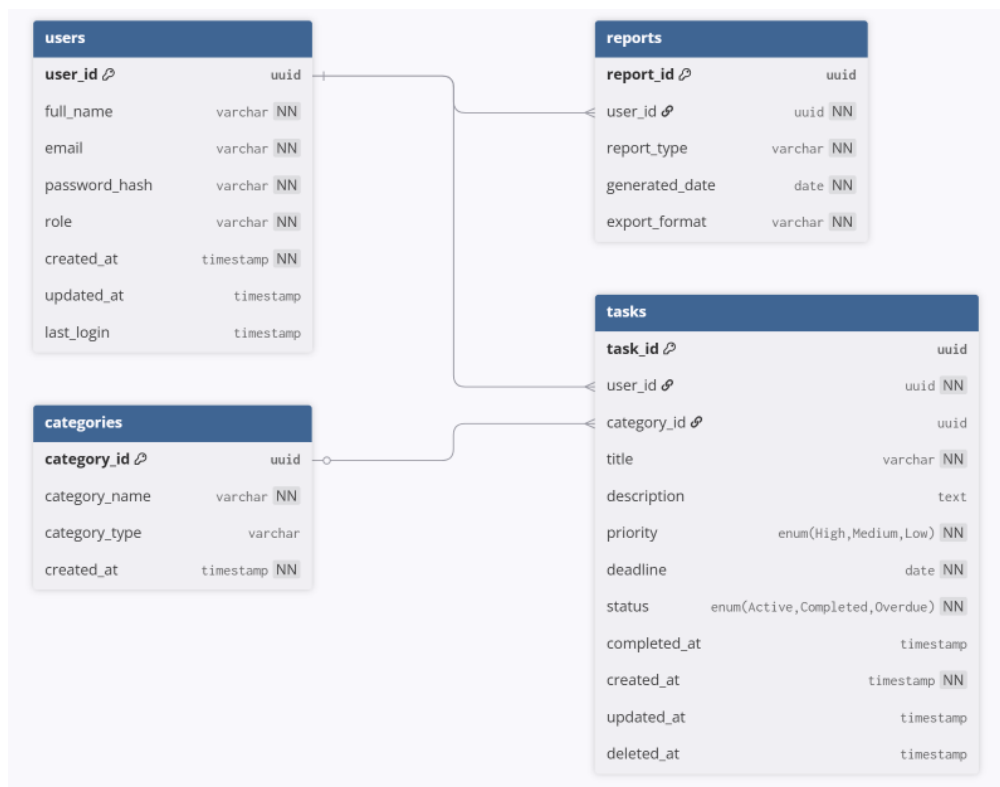
PERANCANGAN DATA

1.6. Entity Relationship Diagram (ERD)

Entity Relationship Diagram (ERD) digunakan untuk menggambarkan struktur basis data sistem DoneRight beserta hubungan antar entitas yang ada di dalam sistem. ERD dirancang berdasarkan kebutuhan fungsional sistem dan disesuaikan dengan Class Diagram pada BAB III sehingga proses implementasi database dapat berjalan secara konsisten. Pada sistem DoneRight, basis data digunakan untuk menyimpan data pengguna, tugas, kategori, dan laporan secara terpusat menggunakan PostgreSQL melalui layanan Supabase. Entitas utama pada sistem terdiri dari:

1. Users
2. Tasks
3. Categories
4. Reports

ERD dirancang agar mampu mendukung seluruh proses utama sistem, seperti autentikasi pengguna, pengelolaan tugas, monitoring admin, visualisasi progress, dan generate laporan.



4.2 Deskripsi Entitas

4.2.1 Entitas Users

Nama Entitas

Users

Deskripsi Entitas

Entitas Users digunakan untuk menyimpan seluruh data akun pengguna sistem, baik mahasiswa maupun admin.

Atribut Entitas Users

Nama Atribut	Tipe Data	Constraint	Keterangan
user_id	UUID	Primary Key	ID unik pengguna
full_name	VARCHAR	NOT NULL	Nama lengkap pengguna
email	VARCHAR	UNIQUE, NOT NULL	Email pengguna
password_hash	VARCHAR	NOT NULL	Password yang telah di-hash
role	VARCHAR	NOT NULL	Role pengguna (mahasiswa/admin)
created_at	TIMESTAMP	NOT NULL	Waktu akun dibuat
updated_at	TIMESTAMP	NULL	Waktu update data
last_login	TIMESTAMP	NULL	Waktu login terakhir

Primary Key

Primary key pada entitas Users adalah user_id.

Relasi Antar Entitas

- Users memiliki relasi dengan Tasks
- Users memiliki relasi dengan Reports

Kardinalitas dan Constraint

- Satu user dapat memiliki banyak task (1:N)
- Satu user dapat menghasilkan banyak report (1:N)

- Email bersifat unik dan tidak boleh sama antar pengguna
- Role hanya dapat bernilai admin atau mahasiswa

4.2.2 Entitas Categories

Nama Entitas

Categories

Deskripsi Entitas

Entitas Categories digunakan untuk menyimpan data kategori tugas yang digunakan untuk pengelompokan task.

Atribut Entitas Categories

Nama Atribut	Tipe Data	Constraint	Keterangan
category_id	UUID	Primary Key	ID kategori
category_name	VARCHAR	NOT NULL	Nama kategori
category_type	VARCHAR	NULL	Jenis kategori
created_at	TIMESTAMP	NOT NULL	Waktu kategori dibuat

Primary Key

Primary key pada entitas Categories adalah category_id.

Relasi Antar Entitas

- Categories memiliki relasi dengan Tasks

Kardinalitas dan Constraint

- Satu kategori dapat digunakan oleh banyak task (1:N)
- category_name tidak boleh kosong
- category_id dapat digunakan sebagai foreign key pada tabel tasks

4.2.3 Entitas Tasks

Nama Entitas

Tasks

Deskripsi Entitas

Entitas Tasks merupakan entitas utama dalam sistem yang digunakan untuk menyimpan seluruh data tugas pengguna.

Atribut Entitas Tasks

Nama Atribut	Tipe Data	Constraint	Keterangan
task_id	UUID	Primary Key	ID tugas
user_id	UUID	Foreign Key	Relasi ke tabel users
category_id	UUID	Foreign Key, NULL	Relasi ke tabel categories
title	VARCHAR	NOT NULL	Judul tugas
description	TEXT	NULL	Deskripsi tugas
priority	ENUM	NOT NULL	Tingkat prioritas
deadline	DATE	NOT NULL	Deadline tugas
status	ENUM	NOT NULL	Status tugas
completed_at	TIMESTAMP	NULL	Waktu tugas selesai
created_at	TIMESTAMP	NOT NULL	Waktu tugas dibuat
updated_at	TIMESTAMP	NULL	Waktu tugas diperbarui
deleted_at	TIMESTAMP	NULL	Penanda soft delete

Primary Key

Primary key pada entitas Tasks adalah task_id.

Relasi Antar Entitas

- Tasks berelasi dengan Users
- Tasks berelasi dengan Categories

Kardinalitas dan Constraint

- Banyak task dimiliki oleh satu user (N:1)
- Banyak task dapat berada pada satu kategori (N:1)
- priority hanya boleh bernilai High, Medium, atau Low
- status hanya boleh bernilai Active, Completed, atau Overdue
- deleted_at digunakan untuk mekanisme soft delete
- category_id dapat bernilai NULL apabila kategori dihapus tetapi tugas dipertahankan
- deadline tidak boleh kosong

4.2.4 Entitas Reports

Nama Entitas

Reports

Deskripsi Entitas

Entitas Reports digunakan untuk menyimpan data laporan yang dihasilkan oleh pengguna maupun admin.

Atribut Entitas Reports

Nama Atribut	Tipe Data	Constraint	Keterangan
report_id	UUID	Primary Key	ID laporan
user_id	UUID	Foreign Key	Relasi ke tabel users
report_type	VARCHAR	NOT NULL	Jenis laporan
generated_date	DATE	NOT NULL	Tanggal generate laporan
export_format	VARCHAR	NOT NULL	Format file laporan

Primary Key

Primary key pada entitas Reports adalah report_id.

Relasi Antar Entitas

- Reports berelasi dengan Users

Kardinalitas dan Constraint

- Banyak report dapat dimiliki oleh satu user (N:1)
- export_format hanya dapat berupa PDF atau CSV
- generated_date tidak boleh kosong

4.3 Relasi Antar Entitas

Entitas	Relasi	Entitas Tujuan	Kardinalitas
Users	memiliki	Tasks	1 : N
Categories	mengelompokkan	Tasks	1 : N
Users	menghasilkan	Reports	1 : N

4.4 Kardinalitas dan Constraint Database

Kardinalitas

1. Users ke Tasks

Satu pengguna dapat memiliki banyak tugas, tetapi satu tugas hanya dimiliki oleh satu pengguna.

2. Categories ke Tasks

Satu kategori dapat digunakan oleh banyak tugas, tetapi satu tugas hanya memiliki satu kategori.

3. Users ke Reports

Satu pengguna dapat menghasilkan banyak laporan.

Constraint Database

Tabel	Constraint
users	email harus unik
users	role hanya admin atau mahasiswa
tasks	priority hanya High, Medium, Low
tasks	status hanya Active, Completed, Overdue
tasks	deleted_at digunakan untuk soft delete
reports	export_format hanya PDF atau CSV

4.5 Konsistensi ERD dengan Class Diagram

ERD pada sistem DoneRight dirancang konsisten dengan Class Diagram pada BAB III sehingga struktur basis data dan struktur class memiliki hubungan yang sinkron.

ERD	Class Diagram
users	User
tasks	Task
categories	Category
reports	Report

Relasi pada ERD juga sesuai dengan relasi association dan inheritance pada Class Diagram:

1. User memiliki banyak Task
2. User menghasilkan banyak Report
3. Category mengelompokkan banyak Task
4. Admin merupakan turunan dari User

BAB V

ARSITEKTUR SISTEM

4

1.7. Gambaran Arsitektur (misal: Web-based / Client–Server)

Sistem DoneRight dikembangkan menggunakan arsitektur *web-based* dengan pola *client-server*. Pada arsitektur ini, pengguna dapat mengakses aplikasi langsung melalui browser tanpa perlu melakukan instalasi perangkat lunak tambahan. Komunikasi antara sisi klien dan server dilakukan menggunakan protokol HTTP/HTTPS dengan pendekatan RESTful API.

Secara umum, arsitektur sistem DoneRight terdiri dari tiga lapisan utama, yaitu *Frontend* sebagai sisi klien, *Backend* sebagai sisi server, dan *Database* sebagai tempat penyimpanan data. Ketiga lapisan tersebut saling terhubung dan bekerja secara terstruktur. Frontend mengirimkan *request* ke Backend melalui REST API, kemudian Backend memproses permintaan tersebut dan melakukan interaksi dengan Database untuk membaca maupun menyimpan data. Setelah proses selesai, Backend mengembalikan *response* dalam format JSON agar dapat ditampilkan kembali pada Frontend kepada pengguna.

Seluruh sistem di-*deploy* menggunakan platform Vercel yang menyediakan layanan *cloud hosting* dengan fitur *continuous deployment*. Dengan penggunaan platform tersebut, sistem dapat diakses secara publik melalui internet tanpa perlu mengelola infrastruktur server fisik secara mandiri oleh tim pengembang.

4.1 Komponen Sistem

a. Frontend

Frontend merupakan bagian antarmuka yang berhubungan langsung dengan pengguna melalui browser. Frontend sistem DoneRight dibangun menggunakan React.js, yaitu library JavaScript berbasis komponen yang digunakan untuk membangun tampilan antarmuka yang interaktif dan responsif. React juga menggunakan konsep Virtual DOM sehingga proses pembaruan tampilan dapat dilakukan lebih efisien tanpa perlu memuat ulang seluruh halaman. Frontend pada sistem DoneRight memiliki beberapa fungsi utama, yaitu:

- 1) Menampilkan halaman registrasi, login, dan dashboard tugas kepada pengguna
- 2) Mengelola state aplikasi secara lokal menggunakan React Hooks seperti `useState` dan `useEffect`
- 3) Mengirimkan permintaan HTTP ke Backend menggunakan Fetch API atau Axios

- 4) Menampilkan data yang diterima dari server, seperti daftar tugas, progress, dan notifikasi
- 5) Melakukan validasi input pada sisi klien sebelum data dikirimkan ke server.

b. Backend

Backend merupakan bagian server yang bertugas menjalankan logika bisnis sistem serta menjadi penghubung antara Frontend dan Database. Komponen ini dikembangkan menggunakan Node.js dengan framework Express.js yang memudahkan proses pembuatan server HTTP dan endpoint API.. Backend pada sistem DoneRight memiliki beberapa fungsi utama, yaitu:

- 1) Menyediakan REST API untuk proses autentikasi, pengelolaan tugas (*CRUD*), pengelolaan kategori, serta monitoring admin
- 2) Melakukan validasi dan sanitasi data yang diterima dari Frontend
- 3) Mengelola proses autentikasi dan otorisasi pengguna menggunakan JSON Web Token (JWT)
- 4) Menjalankan query dan proses pengambilan data melalui Supabase Client SDK
- 5) Mengolah data statistik, seperti presentase progress dan ringkasan produktivitas pengguna.

c. Database

Database merupakan komponen yang digunakan untuk menyimpan seluruh data sistem secara terpusat dan permanen. Sistem DoneRight menggunakan Supabase sebagai layanan *Backend-as-a-Service* (BaaS) yang menyediakan database PostgreSQL berbasis *cloud*. Supabase dipilih karena proses konfigurasi yang relatif mudah, memiliki fitur autentikasi bawaan, serta menyediakan SDK yang mempermudah integrasi dengan Backend berbasis Node.js.

Data yang disimpan dalam database meliputi tabel *users* untuk data akun pengguna, tabel *tasks* untuk data tugas beserta atribut seperti prioritas, *deadline*, status, dan penanda *soft delete*, serta tabel *categories* untuk data kategori tugas yang dikelola admin. Pada sistem ini, data tugas yang dihapus tidak langsung dihapus secara permanen dari database, melainkan hanya ditandai melalui kolom *deleted_at* sebagai penerapan mekanisme *soft delete*.

1.8. Alur komunikasi antar komponen

Komunikasi antar komponen pada sistem DoneRight mengikuti alur request-response berbasis HTTP/HTTPS. Secara umum, alur komunikasi berjalan sebagai berikut:

- 1) Pengguna terlebih dahulu melakukan interaksi melalui antarmuka pada browser, misalnya saat menambahkan tugas atau mengisi formulir tertentu
- 2) Frontend yang dibangun menggunakan React akan melakukan validasi input di sisi klien, kemudian mengirimkan HTTP Request (GET / POST / PATCH / DELETE) beserta data dalam format JSON ke endpoint REST API Backend. Jika pengguna sudah login, setiap request menyertakan JWT token pada header Authorization: Bearer <token>
- 3) Backend yang dibangun menggunakan Express.js kemudian menerima request tersebut untuk diproses lebih lanjut. Pada tahap ini, Backend melakukan verifikasi JWT token, validasi data di sisi server, serta menjalankan query yang diperlukan ke Database melalui Supabase Client SDK
- 4) Database Supabase PostgreSQL akan memproses query yang diterima dan mengembalikan hasilnya ke Backend.
- 5) Backend memformat hasil tersebut menjadi HTTP Response dalam format JSON beserta kode status yang sesuai, seperti *200 OK*, *201 Created*, atau *401 Unauthorized*, lalu mengirimkannya kembali ke Frontend
- 6) Frontend kemudian menerima respons dari server dan memperbarui tampilan antarmuka secara dinamis, misalnya dengan menampilkan daftar tugas terbaru atau memperbarui progress bar tanpa perlu melakukan *refresh* halaman secara penuh

Seluruh komunikasi antara Frontend dan Backend dilakukan menggunakan protokol HTTPS yang terenkripsi sehingga data pengguna lebih aman selama proses transmisi berlangsung. Selain itu, sistem DoneRight di-*deploy* menggunakan platform Vercel sehingga Frontend dan Backend dapat diakses secara publik melalui URL yang disediakan oleh platform tersebut.

1.9. Teknologi yang digunakan

a. React.js

React.js merupakan library JavaScript *open-source* yang digunakan untuk membangun antarmuka pengguna berbasis komponen. Pada sistem DoneRight, React digunakan untuk mengembangkan tampilan sisi klien, seperti halaman registrasi, login, dashboard, dan daftar tugas. React dipilih karena mampu memperbarui tampilan secara reaktif, memiliki ekosistem yang luas, serta mendukung pengembangan aplikasi berbasis *single-page application* (SPA) sehingga perpindahan halaman terasa lebih cepat dan dinamis.

b. Node.js

Node.js merupakan *runtime* JavaScript yang memungkinkan kode JavaScript dijalankan di sisi server. Sementara itu, Express.js adalah framework di atas Node.js yang digunakan untuk membangun REST API dan mengatur logika pada sisi Backend.

Pada sistem DoneRight, Express.js digunakan untuk menangani request dari Frontend, mengatur endpoint API, memverifikasi JWT token, serta menghubungkan sistem dengan database Supabase untuk proses pengolahan data. Penggunaan Express.js dipilih karena strukturnya sederhana, ringan, dan cukup fleksibel untuk pengembangan backend berbasis REST API

c. Supabase

Supabase merupakan platform *Backend-as-a-Service* (BaaS) berbasis *open-source* yang menyediakan database PostgreSQL berbasis *cloud*, autentikasi pengguna, serta API otomatis. Dalam sistem DoneRight, Supabase digunakan sebagai media penyimpanan data utama, seperti data pengguna, data tugas, dan data kategori.

Supabase dipilih karena proses konfigurasi yang relatif mudah, tersedia SDK untuk integrasi dengan Node.js, serta memiliki fitur *Row Level Security* (RLS) yang membantu membatasi akses data antar pengguna sehingga keamanan data lebih terjaga.

d. JSON Web Token

JSON Web Token (JWT) digunakan sebagai mekanisme autentikasi *stateless* pada sistem DoneRight. Setelah pengguna berhasil login, Backend akan menghasilkan token JWT yang berisi informasi seperti *user_id* dan *role* pengguna.

Token tersebut kemudian disimpan di sisi klien dan dikirimkan pada setiap request API melalui *Authorization Header*. Backend akan melakukan verifikasi token sebelum memproses request untuk memastikan identitas dan hak akses pengguna sesuai dengan data yang tersimpan.

e. Vercel

Vercel merupakan platform *cloud deployment* yang digunakan untuk melakukan deployment aplikasi web berbasis React maupun Node.js. Sistem DoneRight di-*deploy* menggunakan Vercel karena integrasinya dengan GitHub cukup mudah serta mendukung *continuous deployment* secara otomatis setiap kali terdapat perubahan kode pada repositori.

Selain itu, Vercel juga menyediakan HTTPS secara default dan mendukung penggunaan *serverless function* sehingga endpoint Backend dapat dijalankan tanpa perlu mengelola server fisik secara manual.

BAB VI

DESAIN ANTARMUKA (UI/UX – Wireframe)

5.1 Prinsip Desain (usability, simplicity, dll)

5.2 Daftar Halaman

- a. Login/Register**
- b. Dashboard**
- c. Fitur utama sistemPrinsip**

5.3 Wireframe (Low-Fidelity)

- a. Sketsa tiap halaman**
- b. Penjelasan fungsi tiap komponen UI**

BAB VII

PROTOTIPE SISTEM (Low-Fidelity)

- 7.1 Deskripsi Prototipe**
- 7.2 User Flow (alur penggunaan)**
 - a. Step-by-step dari perspektif user**
- 7.3 Visualisasi Prototipe**
 - a. Screenshot / sketsa / Figma**
- 7.4 Tools yang Digunakan**

BAB VIII

KONSISTENSI DENGAN SRS (D1) (WAJIB)

8.1 Mapping Kebutuhan

Pada bagian ini dilakukan pemetaan antara kebutuhan fungsional pada dokumen SRS (D1) dengan hasil perancangan sistem pada D2, yang mencakup Use Case Diagram, Activity Diagram, Class Diagram, ERD, dan Wireframe. Hasil pemetaan menunjukkan bahwa seluruh kebutuhan telah direpresentasikan ke dalam model desain sistem.

a. Kebutuhan fungsional → Use Case

Kode	Kebutuhan Fungsional	Implementasi Use Case
FR-01	Registrasi akun pengguna	UC00 Register
FR-02	Autentikasi login	UC01 Login
FR-03	Menambah tugas	UC02 Tambah Tugas
FR-04	Mengedit tugas	UC05 Edit Tugas
FR-05	Soft delete tugas	UC06 Hapus Tugas
FR-06	Menampilkan daftar tugas	UC04 Lihat Daftar Tugas
FR-07	Menandai tugas selesai	UC07 Tandai Tugas Selesai
FR-08	Pencarian tugas	UC08 Cari Tugas
FR-09	Filter tugas	UC09 Filter Tugas
FR-10	Sorting tugas	UC10 Sorting Tugas
FR-11	Menampilkan progress tugas	UC13 Lihat Progress
FR-12	Kelola kategori tugas pengguna	UC18 Kelola Kategori Tugas <i>(perlu penyesuaian logika include/extend)</i>
FR-13	Menampilkan detail tugas	UC04 Lihat Daftar Tugas (detail panel)
FR-14	Penyimpanan data ke database	Tersebar di semua UC (backend process)
FR-15	Pengelompokan berdasarkan kategori	UC04 + UC09
FR-16	Opsi saat hapus kategori	UC18 Kelola Kategori Tugas <i>(extend logic)</i>
FR-17	Monitoring admin	UC15
FR-18	Kelola kategori oleh admin	UC18

FR-19	Monitoring seluruh tugas	UC15
FR-20	Monitoring overdue	UC16
FR-21	Statistik produktivitas	UC17
FR-22	Generate laporan sistem	UC17

b. Proses bisnis → Activity Diagram

Proses Bisnis	Implementasi Activity Diagram
Registrasi pengguna	Activity: Register
Login pengguna	(implicit flow pada autentikasi sistem)
CRUD tugas (Tambah, Edit, Hapus)	Activity: Tambah, Edit, Hapus Tugas
Tandai tugas selesai	Activity: Checklist Selesai
Validasi input	Muncul pada semua proses form (decision node)
Update progress	Parallel flow pada tambah & selesai tugas
Hapus kategori (dengan opsi)	Bagian extend pada UC18 (<i>perlu revisi minor</i>)
Login admin	Activity: Login Admin
Monitoring admin	Activity: Monitoring Sistem

c. Struktur data → Class Diagram & ERD

d. UI → Wireframe

8.2 Validasi Desain

- a. Apakah semua kebutuhan sudah dipetakan ke dalam model perancangan?
- b. Apakah ada kebutuhan yang belum dimodelkan?

BAB IX

PENUTUP

9.1 Kesimpulan

9.2 Rencana Tahap Selanjutnya (Implementasi / D3 – MVP)

LAMPIRAN

1. Semua diagram UML (resolusi besar) – bisa juga berupa link ke draw.io/lucid chart
2. ERD (resolusi besar) – bisa juga berupa link ke draw.io/lucid chart
3. Wireframe lengkap
4. Link prototype (jika ada)